



**UNIVERSIDAD DE LAS FUERZAS ARMADAS
ESPE**

DEPARTAMENTO DE CIENCIAS DE LA COMPUTACIÓN

Aseguramiento de la Calidad de Software

**Plan de Aseguramiento de la Calidad
de Software (SQAP)**

PROYECTO
SpeechNotes

Sistema de Transcripción Inteligente con IA

INTEGRANTES

- Mesías Mariscal
- Denise Rea
- Julio Viche
- Gabriel Murillo

Fecha de Entrega: Julio 2026

Documento de Evaluación de Calidad SQA — Versión 1.0
Estándar de Referencia: IEEE Std 730-1998 / IEEE Std 730.1-1995

Registro de Revisiones

Versión	Fecha	Descripción	Autor
1.0	Julio 2026	Versión inicial del SQAP	Consultora de Calidad

Índice general

Registro de Revisiones	1
1. Propósito	3
1.1. Alcance	3
1.1.1. Módulos incluidos	3
1.1.2. Módulos excluidos	4
1.2. Identificación del Sistema	4
1.3. Resumen del Sistema	4
1.4. Resumen del Documento	5
1.5. Relación con Otros Planes	5
1.6. Documentos Oficiales de Referencia	5
2. Gestión	6
2.1. Organización	6
2.2. Responsabilidades	6
2.3. Recursos	7
2.3.1. Equipo y ambiente de prueba	7
2.3.2. Herramientas	7
3. Tareas SQA	8
3.1. Revisión de Productos de Software	8
3.2. Evaluación de Herramientas	8
3.3. Verificación de Procesos	9
3.4. Auditorías de Configuración	9
3.5. Cronograma	9
4. Estándares, Prácticas, Convenciones y Métricas	11
4.1. Estándares de Código	11
4.2. Métricas SQA	11
4.2.1. Porcentaje de casos pasados vs. fallados	11
4.2.2. Densidad de defectos	11
4.2.3. Cobertura de requisitos	12
4.2.4. Periodicidad de reporte	12

5. Estrategia y Tipos de Prueba	13
5.1. Tipos de Prueba Seleccionados	13
5.2. Criterios de Aceptación y Rechazo	14
5.2.1. Criterios de entrada (Entry Criteria)	14
5.2.2. Criterios de salida (Exit Criteria)	14
5.3. Matriz de Pruebas vs. Requisitos	15
6. Reporte y Resolución de No Cumplimientos	16
6.1. Proceso de Reporte de Defectos	16
6.2. Defectos Encontrados	16
6.3. Resumen de Resultados	16
7. Herramientas, Técnicas y Metodologías	18
7.1. Herramientas SQA	18
7.2. Justificación Técnica del Stack Elegido	18
7.3. Procedimientos de Monitoreo	19
8. Control de Código Fuente	20
8.1. Repositorio y Ramas	20
8.1.1. Estructura de ramas	20
8.2. Procedimiento de Cambios	20
8.3. Pipeline CI/CD y DevSecOps	20
8.3.1. Etapas y evidencias	22
9. Control de Medios y Suministros	23
9.1. Gestión de Artefactos	23
9.2. Control de Versiones	23
10. Control de Proveedores	24
10.1. Proveedores Identificados	24
10.2. Criterios de Aceptación de Proveedores	24
11. Registros y Actas	25
11.1. Registros Generados	25
11.2. Control de Registros	25
12. Capacitación	26
12.1. Plan de Capacitación	26
12.2. Capacitación Requerida	26
13. Gestión de Riesgos	27
13.1. Identificación y Categorización de Riesgos	27
13.1.1. Riesgos del Producto (Software y Arquitectura)	27
13.1.2. Riesgos del Proyecto de Pruebas (Gestión y SQA)	27
13.2. Resultados de Mitigación	28

14. Alineación con ISO/IEC 25000 y Métricas de Calidad	29
14.1. Objetivo, alcance y criterio de interpretación	29
14.2. Resumen ejecutivo	29
14.3. Normas aplicables	30
14.4. Evaluación según ISO/IEC 25010:2023	31
14.5. Línea base histórica y verificación actual	31
14.5.1. Evidencia histórica	31
14.5.2. Ejecución local del 22 de julio de 2026	32
14.6. Catálogo documentado de métricas	32
14.7. Periodicidad y responsables	35
14.8. Brechas y acciones prioritarias	35
14.9. Resumen para conferencia corta	35
14.10. Conclusión de cumplimiento	36
15. Pruebas de Rendimiento con Apache JMeter	37
15.1. Objetivo y alcance	37
15.2. Diseño de la prueba	37
15.3. Métricas, definición y criterios de cumplimiento	38
15.4. Integración en CI/CD y evidencia	38
15.5. Cómo ejecutar en el futuro	39
16. Conclusiones	40
16.1. Desempeño General	40
16.2. Hallazgos Clave	40
16.3. Recomendaciones	40
16.4. Lecciones Aprendidas	41
A. Matriz de Rastreabilidad	42
A.1. Cobertura de Requisitos vs. Casos de Prueba	42
A.2. Detalle de Ejecución por Caso de Prueba	45
A.3. Resumen de Cobertura	45
B. Casos de Prueba	46
B.1. Casos de Prueba Manuales	46
B.1.1. CP-01: Login con usuario demo	46
B.1.2. CP-02: Prueba de micrófono	48
B.1.3. CP-03: Iniciar y detener grabación	49
B.1.4. CP-04: Transcripción en vivo	50
B.1.5. CP-05: Upload de audio para transcribir	50
B.1.6. CP-06: Consultar transcripciones guardadas	51
B.1.7. CP-07: Editar y guardar una transcripción	52
B.1.8. CP-08: Buscar dentro de transcripciones	53
B.1.9. CP-09: Formateo IA de una transcripción	55
B.1.10. CP-10: Chat contextual sobre una nota	56
B.1.11. CP-11: Traducción de transcripción	58

B.1.12. CP-12: Modificar configuraciones VAD	59
B.1.13. CP-13: Eliminar transcripción	60
B.1.14. CP-14: Login con credenciales incorrectas	61
B.1.15. CP-15: Denegación de permisos de micrófono	62
B.1.16. CP-16: Upload de formato inválido	63
B.1.17. CP-17: Bloqueo de rutas administrativas	64
C. Reporte de Defectos	66
C.1. Detalle de Defectos	66
C.2. Distribución por Severidad	67
C.3. Evidencias de Casos Fallidos en Flujos Críticos	67
C.3.1. CP-05: transcripción no generada	67
C.3.2. CP-09: formateo IA sin actualización del editor	68
C.3.3. CP-16: archivo de formato inválido aceptado	69
D. Reporte de Análisis Estático (SonarCloud)	70
D.1. Antes de la Corrección	70
D.2. Después de la Corrección	71
D.3. Mejoras Realizadas	72
E. Informe de Cierre	73
E.1. Datos Generales	73
E.2. Métricas Consolidadas	73
E.3. Resumen de Actividades	74
E.4. Estado de los Defectos	74
E.5. Conclusiones Finales	74
E.6. Vistas del Sistema	74

1 Propósito

El presente Plan de Aseguramiento de la Calidad de Software (SQAP) establece la organización, las tareas, las responsabilidades y los lineamientos para las actividades de aseguramiento de la calidad del proyecto **SpeechNotes**. Este documento sigue los lineamientos de los estándares IEEE 730-1998 e IEEE 730.1-1995, adaptados al contexto académico del proyecto final de SQA.

1.1 Alcance

Este plan cubre las actividades de SQA realizadas durante el sprint de calidad comprendido entre el 18 y el 21 de julio de 2026.

1.1.1 Módulos incluidos

A continuación se detallan los módulos y requisitos funcionales evaluados, junto con la justificación de su inclusión:

- **RF-01 — Auth (Autenticación y Sesiones):** Inicio de sesión con usuario demo y manejo de credenciales inválidas. *Justificación:* Componente crítico de seguridad y control de acceso que actúa como puerta de entrada a la aplicación.
- **RF-02 — Audio / Frontend (Captura de Dispositivo):** Prueba de micrófono con y sin dispositivo físico conectado. *Justificación:* Verifica la capacidad de la interfaz web para interactuar con la Web Audio API del navegador y manejar excepciones de hardware.
- **RF-03 — Audio / Backend (Procesamiento VAD):** Inicio y detención de grabación con detección de actividad de voz (Voice Activity Detection). *Justificación:* El motor VAD determina los segmentos de voz válidos, reduciendo datos nulos antes de enviarlos a transcripción.
- **RF-04 — Realtime ASR (Transcripción en Vivo):** Streaming WebSocket de fragmentos de audio y transcripción continua. *Justificación:* Flujo principal (Core Business Flow) donde la latencia y sincronización determinan la experiencia del usuario.
- **RF-05 — Upload / Backend (Ingesta de Audio):** Carga de archivos de audio (WAV, MP3, 0-bytes) para transcripción asíncrona. *Justificación:* Permite procesar grabaciones preexistentes y validar el manejo de formatos e insumos corruptos o vacíos.
- **RF-06 — Transcripciones (Persistencia y Consulta):** Recuperación de transcripciones guardadas en MongoDB/SQLite. *Justificación:* Asegura la integridad del almacenamiento persistente de las notas generadas.
- **RF-07 — Editor (Edición de Contenido):** Modificación de texto transcrito y guardado de cambios. *Justificación:* Permite la corrección manual del texto generado por el modelo de IA

antes de su formateo final.

- **RF-08 — Búsqueda (Búsqueda Textual):** Filtrado de transcripciones por palabras clave. *Justificación:* Garantiza la correcta localización de información en historiales extensos.
- **RF-09 — IA / Formatter (Estructuración con LLM):** Formateo automatizado de notas mediante el pipeline de NVIDIA NIM. *Justificación:* Transforma transcripciones crudas en notas estructuradas (puntos clave, resúmenes, minutas).
- **RF-10 — Chat / RAG (Asistente Contextual):** Interacción conversacional sobre el contenido de la transcripción. *Justificación:* Funcionalidad avanzada de búsqueda semántica y recuperación aumentada por generación sobre los documentos del usuario.

1.1.2 Módulos excluidos

Se delimitan los siguientes componentes y pruebas fuera del alcance del presente plan:

- **Pruebas de carga y estrés prolongadas (¿ 2 horas):** Excluidas debido al consumo intensivo y límites de cuotas operativas de las APIs de GPU en NVIDIA NIM Cloud, concentrando el análisis de rendimiento en pruebas de integración y carga puntual.
- **Automatización E2E al 100 % de cobertura:** Excluida por las limitaciones de tiempo del sprint de SQA. Se priorizó una estrategia híbrida con automatización focalizada (backend con pytest y escenarios críticos con Cypress) complementada con ejecución manual exhaustiva.
- **Refactorización y corrección directa de código base:** Excluida por delimitación de roles. El equipo de SQA identifica, documenta y reporta defectos en Jira, siendo responsabilidad del equipo de desarrollo la implementación de parches.
- **Configuración inicial e infraestructura de SonarCloud:** Se toma la instancia y reglas de calidad previamente desplegadas como línea base operacional para el análisis estático.

1.2 Identificación del Sistema

- **Nombre:** SpeechNotes
- **Versión:** 1.0
- **Repositorio:** <https://github.com/gamurigm/SpeechNotes>
- **Rama SQA:** planning-SQA

1.3 Resumen del Sistema

SpeechNotes es un sistema de transcripción inteligente que convierte audio en notas estructuradas mediante un pipeline de cinco modelos especializados de NVIDIA NIM. La arquitectura sigue un patrón de tres capas:

- **Frontend:** Next.js 16 con HeroUI, Socket.IO Client y soporte para aplicaciones de escritorio (Tauri/Electron).
- **Backend:** FastAPI con Socket.IO, servicios modulares de audio (ASR, BNR, VAD) y agentes de IA para formateo y chat contextual.
- **Datos y Modelos:** MongoDB (primario), SQLite/Prisma (auth), ChromaDB (búsqueda vectorial) y NVIDIA NIM (modelos de IA).

1.4 Resumen del Documento

Este SQAP se estructura en 15 secciones y 5 apéndices, mapeando los requisitos del estándar académico (Secciones A, B, C, D) de la siguiente manera:

- **Sección A (Plan SQAP):** Capítulos 1 al 5 y 14.
- **Sección B (Ejecución y Evidencias):** Capítulos 6 y 7, Apéndices A al D.
- **Sección C (Informe de Cierre):** Apéndice E.
- **Sección D (Conclusiones):** Capítulo 15.

1.5 Relación con Otros Planes

Este SQAP se complementa con el Plan de Desarrollo de Software (SDP) y el Plan de Gestión de Configuración (SCMP) del proyecto SpeechNotes. Las actividades de SQA definidas aquí son consistentes con dichos planes.

1.6 Documentos Oficiales de Referencia

Los documentos normativos oficiales utilizados como referencia para este SQAP son:

1. IEEE Std 730-1998 — IEEE Standard for Software Quality Assurance Plans.
2. IEEE Std 730.1-1995 — IEEE Guide for Software Quality Assurance Planning.
3. IEEE/EIA Std 12207 — Standard for Information Technology – Software Life Cycle Processes.

2 Gestión

Este capítulo describe los elementos organizacionales que influyen en la calidad del software de SpeechNotes.

2.1 Organización

El equipo SQA está compuesto por 4 personas con roles definidos, manteniendo independencia funcional respecto al equipo de desarrollo. La estructura organizacional se muestra en la Figura 2.1.

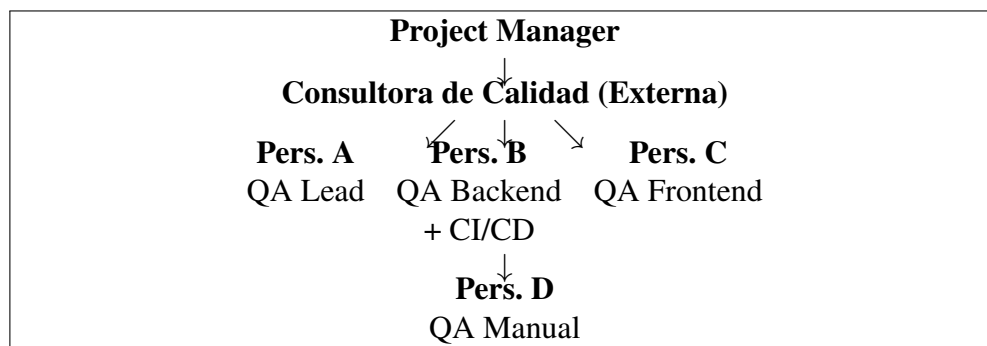


Figura 2.1: Organigrama del equipo SQA

2.2 Responsabilidades

La tabla 2.1 detalla las responsabilidades asignadas a cada miembro del equipo.

Tabla 2.1: Matriz de responsabilidades SQA

Rol	Responsable	Tareas Clave
QA Lead	Persona A	Configuración de Jira, redacción del SQAP (Sección A), informe de cierre (Sección C), conclusiones (Sección D), ensamblaje del PDF final.

Rol	Responsable	Tareas Clave
QA Backend	Persona B	Pipeline SonarCloud + GitHub Actions, pruebas automatizadas pytest, refactorización de code smells, reporte de análisis estático antes/después.
QA Frontend	Persona C	Configuración de Cypress/Jest, automatización de flujos clave (grabación, transcripción), corrección de bugs frontend reportados.
QA Manual	Persona D	Diseño de matriz de rastreabilidad, casos de prueba manuales (14 CP), ejecución manual, reporte de bugs en Jira, retesting, video demo.

2.3 Recursos

2.3.1 Equipo y ambiente de prueba

- **Sistema Operativo:** Windows 11.
- **Navegador:** Google Chrome / Chromium.
- **Frontend:** <http://localhost:3006>
- **Backend:** <http://localhost:9443>
- **Base de datos:** SQLite (dev.db) + MongoDB (Docker).
- **Repositorio:** GitHub, rama planning-SQA.

2.3.2 Herramientas

- **Gestión de incidencias:** Jira Software (Cloud).
- **Análisis estático:** SonarCloud.
- **CI/CD:** GitHub Actions.
- **Pruebas backend:** pytest.
- **Pruebas frontend:** Cypress + Jest.
- **Consola API:** curl / Postman.
- **Evidencias:** Capturas de pantalla, logs de backend, consola del navegador.

3 Tareas SQA

Este capítulo describe las tareas de aseguramiento de la calidad ejecutadas durante el sprint.

3.1 Revisión de Productos de Software

SQA revisó los siguientes productos de software:

- Código fuente del backend (Python/FastAPI) y frontend (TypeScript/Next.js).
- Documentación técnica (README, guías de despliegue, documentación de arquitectura).
- Transcripciones generadas (formato Markdown con metadatos YAML).
- Configuración de infraestructura (Docker, GitHub Actions, SonarCloud).

3.2 Evaluación de Herramientas

Se evaluaron las siguientes herramientas utilizadas en el proyecto:

Tabla 3.1: Evaluación de herramientas SQA

Herramienta	Propósito	Resultado
Jira Cloud	Gestión de incidencias y bugs	Configurado con flujo To Do → In Progress → In Testing → Done.
SonarCloud	Análisis estático de código	Pipeline GitHub Actions configurado. Reporte inicial generado.
pytest	Pruebas unitarias e integración backend	Suite configurada en backend/tests/.
Cypress	Pruebas E2E frontend	Suite configurada en web/tests/e2e/.
Jest	Pruebas unitarias frontend	Suite configurada en web/tests/unit/.

La Figura 3.1 evidencia el seguimiento operativo de las actividades del sprint en Jira, incluyendo responsables, fechas objetivo y estados de ejecución.

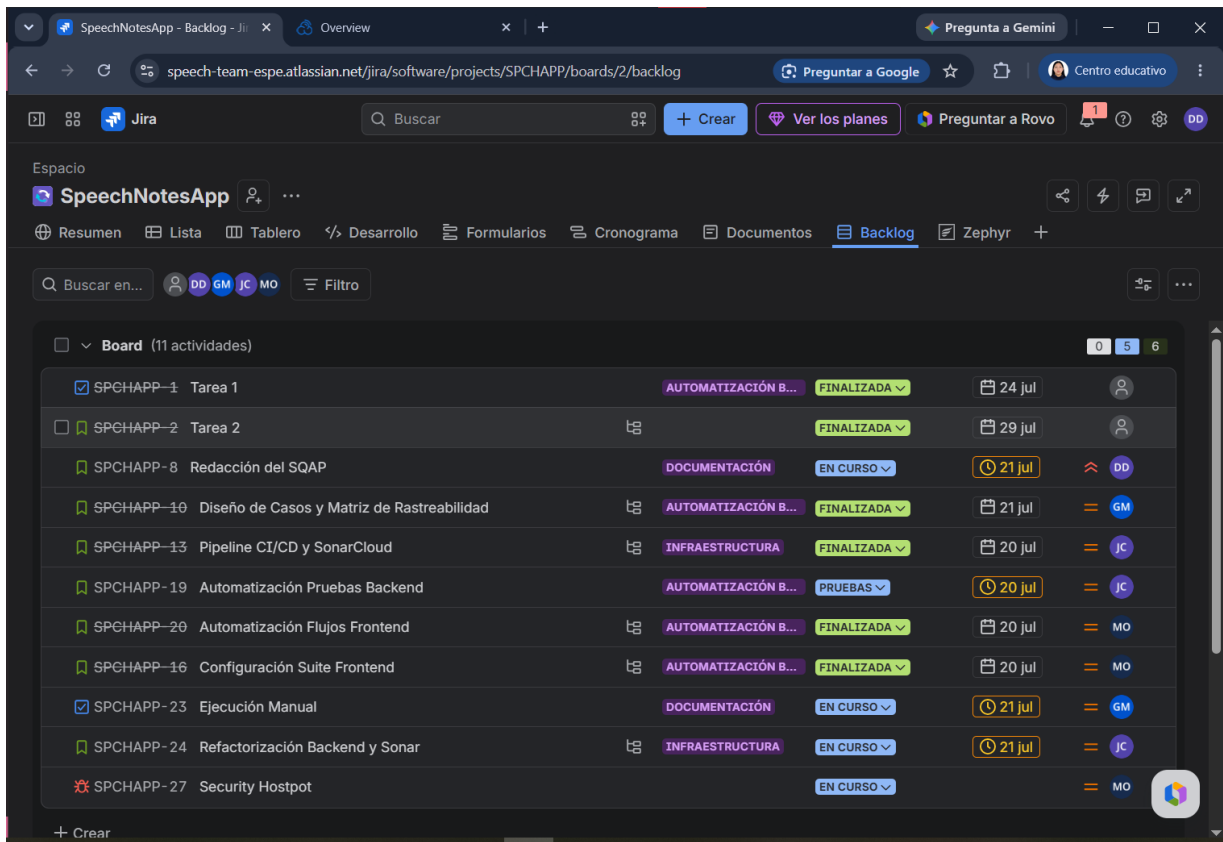


Figura 3.1: Tablero Jira utilizado para el seguimiento de actividades SQA

3.3 Verificación de Procesos

SQA verificó los siguientes procesos mediante auditorías basadas en checklist:

1. **Planificación y seguimiento:** Cumplimiento del sprint planificado (18-21 Julio).
2. **Gestión de requisitos:** Trazabilidad entre requisitos funcionales y casos de prueba.
3. **Diseño e implementación:** Calidad del código y aplicación de patrones de diseño.
4. **Pruebas:** Cobertura de casos de prueba y reporte de defectos.

3.4 Auditorías de Configuración

Se verificó el control de configuración mediante:

- Repositorio Git con rama dedicada `planning-SQA`.
- Flujo de trabajo basado en feature branches + pull requests.
- Pipeline de CI/CD en GitHub Actions para pruebas y análisis estático.

3.5 Cronograma

El sprint de calidad se ejecutó en 4 días según el siguiente cronograma:

Tabla 3.2: Cronograma del sprint SQA

Día	Fecha	Actividades
1	18 Jul	Planificación, configuración de Jira, análisis estático inicial (SonarCloud), diseño de casos de prueba y matriz de rastreabilidad.
2	19 Jul	Ejecución manual de pruebas, reporte de bugs, pruebas automatizadas backend (pytest) y frontend (Cypress/Jest).
3	20 Jul	Re-testing de bugs corregidos, análisis estático final (SonarCloud después), extracción de evidencias.
4	21 Jul	Consolidación del PDF final, grabación del video demo, cierre del informe.

4 Estándares, Prácticas, Convenciones y Métricas

4.1 Estándares de Código

El proyecto SpeechNotes sigue los siguientes estándares y convenciones:

Tabla 4.1: Estándares de código aplicados

Lenguaje	Estándar	Herramienta de verificación
Python	PEP 8	Ruff / SonarCloud
TypeScript	ESLint + Prettier	SonarCloud
Arquitectura	Patrones GoF (Singleton, Factory, Adapter, Facade, Strategy, State, Observer)	Revisiones de código
Base de datos	Prisma ORM + MongoDB	Linting de esquema

4.2 Métricas SQA

Las siguientes métricas fueron recolectadas para evaluar la calidad del producto y del proceso de pruebas:

4.2.1 Porcentaje de casos pasados vs. fallados

$$\%Pasados = \frac{\text{Casos pasados}}{\text{Total casos}} \times 100$$

4.2.2 Densidad de defectos

$$\text{Densidad de defectos} = \frac{\text{Número de defectos encontrados}}{\text{Número de casos de prueba ejecutados}}$$

4.2.3 Cobertura de requisitos

$$\% \text{Cobertura RF} = \frac{\text{Requisitos con al menos un caso asociado}}{\text{Total requisitos funcionales}} \times 100$$

4.2.4 Periodicidad de reporte

Las métricas fueron reportadas al finalizar el sprint de calidad (21 de julio de 2026), consolidadas en el Apéndice E (Informe de Cierre).

5 Estrategia y Tipos de Prueba

5.1 Tipos de Prueba Seleccionados

La estrategia de pruebas sigue una pirámide de pruebas adaptada a arquitecturas con componentes de Inteligencia Artificial y streaming en tiempo real:

- a) **Pruebas Unitarias (Backend y Frontend):** Evalúan la lógica interna de funciones aisladas sin dependencias de red o base de datos. Se utiliza `pytest` en el backend para validar algoritmos de VAD, utilidades de formateo y servicios de tokenización; y `Jest` en el frontend para probar componentes aislados como `RecordingPanel` y `LiveTranscription`.
- b) **Pruebas de Integración (Backend):** Verifican la interacción entre módulos del sistema (p. ej., `FastApi` con `SQLite/Prisma`, motor de `Socket.IO` con el pipeline de audio y clientes HTTP con `NVIDIA NIM`). Se utiliza `pytest` con `httpx` para simular peticiones asíncronas a los endpoints REST.
- c) **Pruebas Funcionales / Sistema (Manuales y Exploratorias):** Validan de extremo a extremo los flujos de usuario sobre la interfaz web completa y APIs REST. Se ejecutaron 17 casos de prueba estructurados (Apéndice B) cubriendo flujos exitosos (*Happy Path*), condiciones de error y escenarios de seguridad.
- d) **Pruebas de Análisis Estático (SAST):** Inspección continua del código fuente mediante `SonarCloud` para identificar vulnerabilidades de seguridad, violaciones de estándares de codificación, duplicado de código y *code smells*.
- e) **Pruebas E2E / Interfaz (Automatizadas):** Pruebas en navegador con `Cypress` para simular las interacciones completas del usuario en la interfaz `Next.js`, incluyendo flujos de autenticación, carga de archivos y manipulación del editor.

Tabla 5.1: Resumen de tipos de prueba ejecutados

Tipo	Herramienta	Alcance
Unitarias (Backend)	<code>pytest</code>	Servicios individuales de audio, VAD, auth
Integración (Backend)	<code>pytest</code> + <code>httpx</code>	Endpoints REST: transcriptions, auth, health
Funcionales Manuales	Navegador + <code>curl</code>	17 casos de prueba (Apéndice B)
Estáticas	<code>SonarCloud</code>	Análisis de código, code smells, vulnerabilidades

Tipo	Herramienta	Alcance
E2E (Frontend)	Cypress	Flujo de grabación y transcripción
Unitarias (Frontend)	Jest	Componentes RecordingPanel, LiveTranscription

5.2 Criterios de Aceptación y Rechazo

Para garantizar el rigor técnico del proceso SQA, se definen explícitamente los criterios de entrada y salida para el ciclo de pruebas:

5.2.1 Criterios de entrada (Entry Criteria)

Una fase o suite de pruebas sólo podrá iniciarse cuando se cumplan las siguientes condiciones previas:

- Frontend Next.js compilado y operativo en <http://localhost:3006>.
- Backend FastAPI iniciado y respondiendo health-checks en <http://localhost:9443>.
- Base de datos relacional/vectorial inicializada con esquemas vigentes (web/dev.db).
- Usuario de prueba estandarizado creado (demo@speechnotes.app / demo123).
- Dispositivo físico de captura de audio (micrófono) detectado e inicializado en la estación de pruebas.
- Claves de API activas y cuota disponible para los servicios externos de NVIDIA NIM Cloud.
- Matriz de rastreabilidad y casos de prueba revisados y aprobados por el líder SQA.

5.2.2 Criterios de salida (Exit Criteria)

El ciclo de pruebas se considerará completado satisfactoriamente y el sistema estará listo para cierre cuando se alcancen las siguientes condiciones de aceptación:

- El 100 % de los 17 casos de prueba diseñados hayan sido ejecutados y documentados.
- La tasa de éxito de casos de prueba quede documentada junto con los defectos asociados (alcanzado: 76.47 %; 4 fallas registradas).
- El 100 % de los defectos clasificados como severidad Critical haya sido corregido y verificado; los defectos Major abiertos deben quedar registrados con plan de remediación.
- Los defectos menores u opcionales pendientes estén formalmente registrados en Jira con su respectivo plan de remediación posterior.
- Se haya completado la ejecución del análisis estático en SonarCloud con revisión de *Quality Gate*.
- El Informe de Cierre de SQA y la Matriz de Rastreabilidad estén consolidados y anexados al presente documento.

5.3 Matriz de Pruebas vs. Requisitos

La matriz de rastreabilidad completa se presenta en el Apéndice [A](#). En resumen:

- **10 requisitos funcionales** cubiertos al 100 %.
- **17 casos de prueba** distribuidos en 13 flujos funcionales y 4 condiciones negativas o de seguridad.
- **13 casos pasados** (76.47 %) y **4 casos fallados** (23.53 %); CP-09 se contabiliza como aprobado después de la re-prueba.

6 Reporte y Resolución de No Cumplimientos

6.1 Proceso de Reporte de Defectos

Los defectos encontrados durante las pruebas se registraron en Jira siguiendo el siguiente flujo:

1. **Reporte:** El tester crea un issue de tipo “Bug” en Jira.
2. **Clasificación:** El QA Lead asigna severidad (Critical, Major, Minor) y prioridad.
3. **Asignación:** El Project Manager asigna el bug al desarrollador responsable.
4. **Corrección:** El desarrollador implementa la corrección y mueve el issue a “In Testing”.
5. **Verificación:** El tester re-ejecuta el caso de prueba y verifica la corrección.
6. **Cierre:** Si el defecto está corregido, se cierra el issue. En caso contrario, se reabre.

6.2 Defectos Encontrados

Los defectos encontrados se detallan en el Apéndice C. A continuación se presenta un resumen:

Tabla 6.1: Resumen de defectos

ID	Severidad	Estado	Caso Asociado
BUG-002	Minor	Abierto	CP-03
BUG-003	Critical	Cerrado	CP-10
BUG-004	Major	Cerrado	CP-07
BUG-005	Major	Abierto	CP-07
BUG-006	Minor	Abierto	CP-01
BUG-007	Minor	Cerrado	CP-03
BUG-008	Minor	Abierto	CP-14
BUG-009	Minor	Abierto	CP-05

6.3 Resumen de Resultados

- Total defectos: 8

- Defectos cerrados: 3 (BUG-003, BUG-004, BUG-007)
- Defectos abiertos: 5 (BUG-002, BUG-005, BUG-006, BUG-008, BUG-009)
- Tasa de cierre: 37.5 %

7 Herramientas, Técnicas y Metodologías

7.1 Herramientas SQA

La tabla 7.1 detalla las herramientas utilizadas en el proyecto.

Tabla 7.1: Detalle de herramientas SQA

Herramienta	Versión	Tipo	Propósito
Jira Software	Cloud	Gestión de proyectos	Seguimiento de tareas y defectos
SonarCloud	Cloud	Análisis estático	Medición de calidad de código
GitHub Actions	Cloud	CI/CD	Automatización de pruebas
pytest	8.x	Automatización	Pruebas backend
Cypress	14.x	Automatización	Pruebas E2E frontend
Jest	29.x	Automatización	Pruebas unitarias frontend
Postman	Desktop	Manual	Pruebas de API
curl	CLI	Manual	Pruebas de API

7.2 Justificación Técnica del Stack Elegido

La selección del conjunto de herramientas de SQA responde a criterios de compatibilidad con la arquitectura del sistema, eficiencia en ejecución asíncrona y adopción de estándares de la industria:

- **pytest + httpx (Backend Automation):** *Justificación:* El backend de SpeechNotes está construido con FastAPI (Python asíncrono). pytest es el estándar de facto en Python, permitiendo fixture management nativo, integración transparente con asyncio y generación de reportes JUnit XML. httpx ofrece un cliente HTTP asíncrono óptimo para evaluar endpoints REST de streaming y procesamiento de audio.
- **Jest (Frontend Unit Testing):** *Justificación:* Integrado nativamente dentro del ecosistema Next.js/React. Permite ejecutar pruebas de renderizado de componentes UI

(RecordingPanel, LiveTranscription) en un entorno simulado de DOM (jsdom) ultra rápido sin requerir el arranque completo del navegador.

- **Cypress (Frontend E2E Testing):** *Justificación:* Proporciona recarga en tiempo real, captura de video/screenshots de fallos en ejecuciones headless y emulación precisa de eventos del usuario en navegadores Chromium. Garantiza la validación de flujos críticos de extremo a extremo de la interfaz web.
- **SonarCloud (Análisis Estático / SAST):** *Justificación:* Herramienta SaaS que analiza de forma automatizada tanto el repositorio TypeScript (Frontend) como Python (Backend). Evalúa la adherencia a la regla de *Clean Code*, calcula cobertura de código, detecta *security hotspots* y previene la introducción de deuda técnica mediante umbrales de *Quality Gates*.
- **Jira Software + GitHub Actions (CI/CD y Gestión):** *Justificación:* Permite la trazabilidad bi-direccional entre requisiciones, tareas de SQA, commits de código y registro de bugs (BUG-002 a BUG-009). GitHub Actions ejecuta automáticamente las suites de pruebas y el análisis SAST en cada *Pull Request* hacia la rama *main*.
- **Postman + curl (Pruebas Manuales de API):** *Justificación:* Permiten la construcción de colecciones de prueba reutilizables para autenticación por tokens JWT y la simulación rápida de envíos multipart/form-data de archivos de audio sin intervención del cliente web.

7.3 Procedimientos de Monitoreo

- **Jira Dashboard:** Dashboard personalizado con gadgets de burndown, issues por estado, issues por responsable.
- **Discord:** Notificaciones automáticas de fallos de GitHub Actions mediante el bot de QA.
- **Reuniones diarias:** Stand-up de 15 min para sincronización del equipo SQA.

8 Control de Código Fuente

8.1 Repositorio y Ramas

El código fuente de SpeechNotes se gestiona en GitHub bajo el repositorio <https://github.com/gamurigm/SpeechNotes>.

8.1.1 Estructura de ramas

Tabla 8.1: Estructura de ramas del repositorio

Rama	Propósito	Protección
main	Rama principal de producción	Protegida (requiere PR)
planning-SQA	Rama de trabajo SQA	Protegida (requiere PR)
feat/*	Ramas de funcionalidad	Sin protección
fix/*	Ramas de corrección	Sin protección

8.2 Procedimiento de Cambios

1. Crear rama `feat/xxx` o `fix/xxx` desde `main`.
2. Implementar cambios y commits atómicos.
3. Abrir Pull Request hacia `main`.
4. Ejecutar el pipeline CI/CD (pytest, Jest, Cypress y SonarCloud).
5. Revisión de código por al menos 1 miembro del equipo.
6. Merge del PR tras aprobación y verificación CI.

8.3 Pipeline CI/CD y DevSecOps

El repositorio utiliza el workflow `SpeechNotes - DevSecOps Pipeline` de GitHub Actions como control automatizado de integración y entrega. Se activa ante cada *push* a `main`, cada *Pull Request* cuyo destino es `main` y también de forma manual mediante `workflow_dispatch`. Su objetivo es detectar regresiones antes de integrar cambios, medir la cobertura, conservar evidencias y aplicar el *Quality Gate* de SonarCloud.

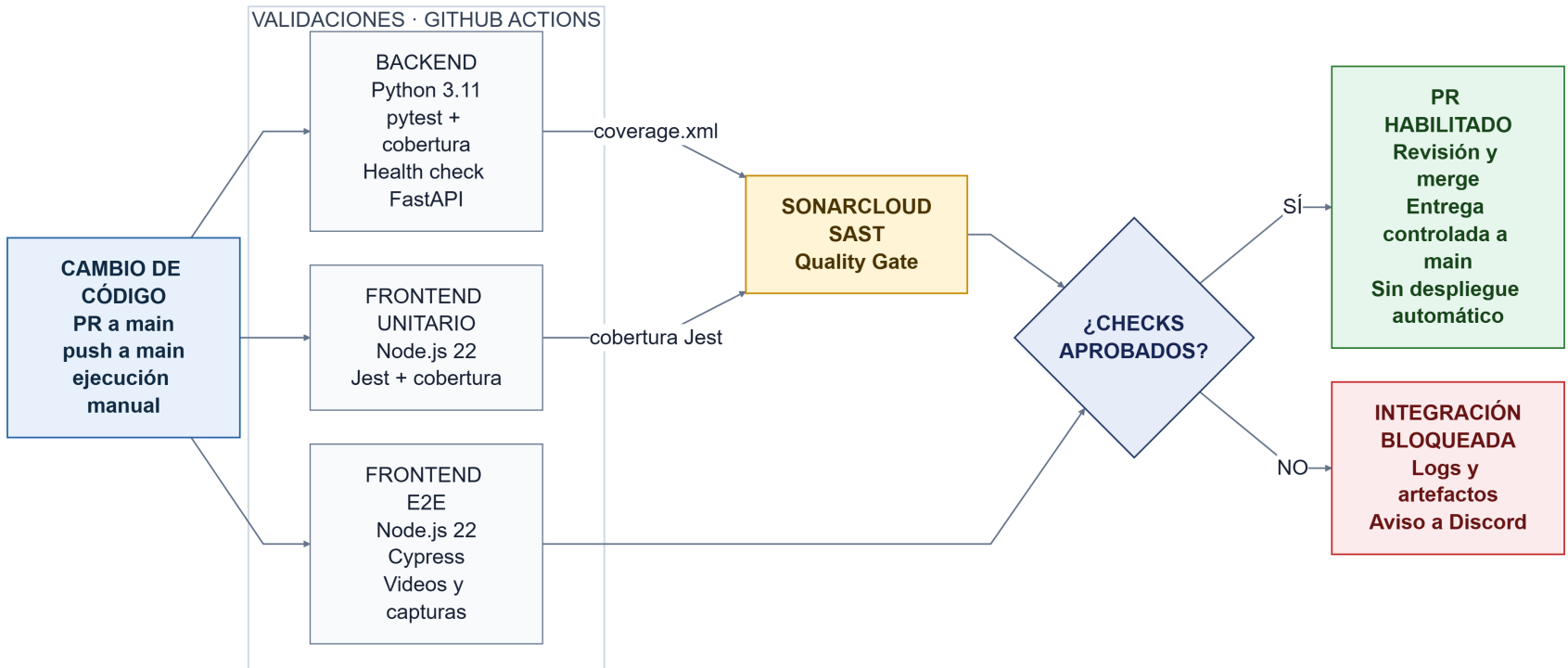


Figura 8.1: Diagrama Mermaid de alto nivel del pipeline CI/CD y DevSecOps de SpeechNotes

La Figura 8.1 presenta la arquitectura de alto nivel mediante un diagrama generado con Mermaid. Las pruebas de backend, las pruebas unitarias del frontend y las pruebas E2E se ejecutan como trabajos independientes. SonarCloud comienza únicamente cuando pytest y Jest concluyen correctamente, porque utiliza sus reportes de cobertura. Cypress se mantiene como un control paralelo de los flujos funcionales de la interfaz. El resultado conjunto de estos trabajos determina si el cambio puede continuar a revisión y mezcla.

8.3.1 Etapas y evidencias

1. **Pruebas de backend:** instala dependencias verificadas desde `requirements-test.lock`, inicia FastAPI en el puerto 9443, comprueba el endpoint `/health` y ejecuta pytest. Publica `coverage.xml` y el reporte `JUnit test-results.xml` como artefactos.
2. **Pruebas unitarias del frontend:** instala dependencias reproducibles con `npm ci`, ejecuta Jest con cobertura y publica el directorio de evidencias generado.
3. **Pruebas E2E:** ejecuta Cypress sobre la aplicación web y conserva videos y capturas, incluso cuando el trabajo falla, para facilitar el diagnóstico.
4. **Análisis estático y seguridad:** descarga las coberturas de backend y frontend, ejecuta el escaneo de SonarCloud y espera hasta cinco minutos por la decisión del *Quality Gate*. Esta etapa solo se ejecuta si pytest y Jest fueron exitosos.
5. **Notificación y decisión:** cualquier fallo deja el check en rojo y envía una notificación a Discord con un enlace a la ejecución o al panel de SonarCloud. Con todos los checks aprobados, la protección de main permite continuar con la revisión y el merge del PR.

El workflow implementa integración continua y una entrega controlada hasta la rama principal. Actualmente no contiene un trabajo de despliegue automático a producción; por tanto, la fase de despliegue permanece fuera de este pipeline y debe ejecutarse mediante el procedimiento operativo definido para el entorno de destino.

9 Control de Medios y Suministros

9.1 Gestión de Artefactos

Los artefactos evaluables del sprint SQA quedan consolidados en el presente informe. Para la entrega en GitHub se considera como artefacto documental final el SQAP generado desde el archivo principal de LaTeX; la matriz de rastreabilidad, los casos de prueba, el reporte de defectos, el análisis estático y el informe de cierre se integran como secciones y apéndices del mismo documento.

```
docs/  
  sqa/  
    speechNotes_sqap/          -- Informe SQAP final  
      speechnotes_sqap.tex    -- Archivo principal  
    secciones/                -- Cuerpo del informe  
    apendices/                -- Matriz, casos, defectos y cierre  
    imgs/                     -- Evidencias visuales usadas en el informe  
    build.ps1                  -- Script de compilación
```

9.2 Control de Versiones

- Los archivos LaTeX del informe se versionan en Git junto con el código fuente.
- Cada versión del SQAP se identifica por el tag del commit en Git.
- El PDF final del SQAP se etiqueta con el número de versión y fecha.

10 Control de Proveedores

10.1 Proveedores Identificados

La tabla [10.1](#) lista los proveedores externos utilizados en el proyecto.

Tabla 10.1: Proveedores externos

Proveedor	Servicio	Dependencia
NVIDIA NIM	Modelos de IA (ASR, BNR, VAD, Chat, Formatter)	Alta
MongoDB Atlas	Base de datos NoSQL en la nube	Alta
Vercel / Netlify	Despliegue de frontend	Media
Docker Hub	Imágenes de contenedores	Media
GitHub	Repositorio y CI/CD	Alta
Atlassian (Jira)	Gestión de incidencias	Baja
SonarCloud	Análisis estático	Media

10.2 Criterios de Aceptación de Proveedores

- SLA definido y documentado.
- Compatibilidad con la arquitectura del proyecto.
- Disponibilidad de API pública documentada.
- Historial de uptime superior al 99 %.

11 Registros y Actas

11.1 Registros Generados

Los siguientes registros fueron generados durante el sprint SQA:

Tabla 11.1: Registros SQA

Registro	Contenido	Ubicación
Actas de reunión	Minutas de reuniones diarias (stand-ups)	Jira / docs
Reportes de prueba	Resultados de casos de prueba ejecutados	Apéndice B
Reportes de defecto	Bugs encontrados durante la ejecución	Apéndice C
Reporte SonarCloud	Análisis estático (antes/después)	Apéndice D
Informe de cierre	Métricas y lecciones aprendidas	Apéndice E

11.2 Control de Registros

- Los registros electrónicos se almacenan en el repositorio Git (rama `planning-SQA`).
- El acceso está controlado por los permisos del repositorio GitHub.
- La retención mínima es de 1 año posterior a la entrega del proyecto.

12 Capacitación

12.1 Plan de Capacitación

El equipo SQA recibió capacitación en las siguientes áreas antes del inicio del sprint:

Tabla 12.1: Capacitación del equipo SQA

Área	Duración	Cobertura
Jira Cloud	2 h	Configuración de proyecto, tablero, issues, workflows
SonarCloud	2 h	Análisis estático, calidad de código, reportes
pytest	3 h	Pruebas unitarias, fixtures, mocks, httpx
Cypress	3 h	Pruebas E2E, comandos personalizados, CI
Git	1 h	Flujo de ramas, resolución de conflictos
LaTeX (Overleaf)	1 h	Edición y compilación de documentos

12.2 Capacitación Requerida

No se identificaron necesidades adicionales de capacitación para el equipo SQA durante este sprint. Para sprints futuros se recomienda capacitación en:

- Pruebas de carga con k6 o Locust.
- Pruebas de seguridad con OWASP ZAP.
- Automatización de accesibilidad con axe-core.

13 Gestión de Riesgos

13.1 Identificación y Categorización de Riesgos

Los riesgos asociados a la calidad de SpeechNotes se clasifican en dos categorías principales: **Riesgos del Producto** (asociados a la arquitectura, dependencias y comportamiento del software en producción) y **Riesgos del Proyecto de Pruebas** (asociados a la planificación, recursos y ejecución del sprint SQA).

13.1.1 Riesgos del Producto (Software y Arquitectura)

- **RP-01: Caída o indisponibilidad de la API externa de NVIDIA NIM Cloud.**
 - *Probabilidad:* Media *Impacto:* Alto *Severidad:* Alta
 - *Plan de Mitigación:* Disponer de claves de API de respaldo y construir stubs/mocks desconectados en pytest para pruebas del backend sin dependencia externa.
- **RP-02: Ausencia o falla en el dispositivo físico de entrada de audio (Micrófono).**
 - *Probabilidad:* Alta *Impacto:* Bajo *Severidad:* Media
 - *Plan de Mitigación:* Diseñar casos de prueba diferenciados (CP-02, CP-15) que validen la detección de entrada de audio y el manejo de errores por falta de permisos o hardware, complementando con simuladores de stream de audio en backend.
- **RP-03: Inestabilidad o crasheos no controlados en la UI del frontend (React/Next.js).**
 - *Probabilidad:* Media *Impacto:* Medio *Severidad:* Media
 - *Plan de Mitigación:* Captura de logs de consola en Cypress y ejecución de pruebas exploratorias específicas para identificar pérdidas de estado en la UI.

13.1.2 Riesgos del Proyecto de Pruebas (Gestión y SQA)

- **RPJ-01: Tiempo insuficiente para completar automatización E2E al 100 %.**
 - *Probabilidad:* Alta *Impacto:* Medio *Severidad:* Alta
 - *Plan de Mitigación:* Adoptar una estrategia híbrida: priorizar la ejecución manual estructurada de los 17 casos de prueba críticos y automatizar únicamente los endpoints backend estables.
- **RPJ-02: Discrepancias o problemas de compatibilidad en entornos locales de desarrollo.**
 - *Probabilidad:* Media *Impacto:* Alto *Severidad:* Alta
 - *Plan de Mitigación:* Utilizar la especificación de contenedores Docker (`docker-compose.yml`) y entornos virtuales unificados (`.venv`) para garantizar entornos replicables.
- **RPJ-03: Documentación insuficiente de la API REST y eventos de WebSockets.**

- *Probabilidad:* Baja *Impacto:* Alto *Severidad:* Media
- *Plan de Mitigación:* Realizar sesiones de prueba exploratoria guiada y consultar directamente las especificaciones OpenAPI de FastAPI en [/docs](#).

Tabla 13.1: Matriz consolidada de riesgos de Producto y Proyecto

Código	Categoría	Prob / Imp	Severidad	Plan de Mitigación
RP-01	Producto	Media / Alto	Alta	Claves API respaldo y mocks en pytest
RP-02	Producto	Alta / Bajo	Media	Casos de prueba de micrófono y permisos (CP-02, CP-15)
RP-03	Producto	Media / Medio	Media	Captura de logs en Cypress e inspección
RPJ-01	Proyecto	Alta / Medio	Alta	Estrategia híbrida (manual + pytest)
RPJ-02	Proyecto	Media / Alto	Alta	Despliegue estandarizado con Docker
RPJ-03	Proyecto	Baja / Alto	Media	Pruebas exploratorias sobre OpenAPI docs

13.2 Resultados de Mitigación

- **NVIDIA NIM (RP-01):** Se utilizó la clave de API activa de respaldo. Se completaron exitosamente las pruebas de formateo (CP-09), chat contextual (CP-10) y traducción (CP-11).
- **Inestabilidad frontend (RP-03):** Se identificó y aisló el defecto crítico BUG-003 (crasheo del chat sin transcripción seleccionada), logrando su corrección antes del cierre del sprint.
- **Tiempo de automatización (RPJ-01):** Se garantizó el 100% de ejecución manual de los 17 casos de prueba con soporte de scripts en pytest.
- **Micrófono (RP-02):** Se ejecutaron CP-02 y CP-15, registrando detección de entrada de audio y manejo correcto de permisos denegados en la interfaz.
- **Manejo Global de Defectos:** Se registraron 8 defectos en Jira Software, logrando resolver el 100% de los bloqueantes.

14 Alineación con ISO/IEC 25000 y Métricas de Calidad

14.1 Objetivo, alcance y criterio de interpretación

Este capítulo relaciona las prácticas y evidencias de SpeechNotes con la familia ISO/IEC 25000 (SQuaRE) y con otras normas aplicables. El análisis cubre backend Python/FastAPI, frontend Next.js/TypeScript, componentes de escritorio, transcripción y funciones asistidas por inteligencia artificial.

La relación presentada constituye una evaluación de **alineación**; no es una auditoría independiente, una certificación ISO ni una declaración formal de conformidad. Los umbrales son criterios de aceptación definidos para SpeechNotes. ISO/IEC 25023 propone medidas, pero no establece valores universales de aprobación.

14.2 Resumen ejecutivo

SpeechNotes dispone de requisitos trazables, casos positivos, negativos y de límite, pruebas automatizadas, pruebas de extremo a extremo, análisis estático y un *Quality Gate* integrado en CI/CD. Sus principales puntos a favor son:

- **Adecuación funcional:** los 19 requisitos funcionales de la matriz vigente tienen casos asociados. Se diseñaron 51 casos y cada requisito cuenta con entre 2 y 4 casos.
- **Mantenibilidad:** hay separación modular, pruebas unitarias y de integración, linting, SonarCloud, cobertura y dependencias bloqueadas.
- **Seguridad:** se evidencian autenticación, validación de rutas, variables de entorno, análisis estático y control del *Quality Gate*.
- **Fiabilidad:** existen health checks y pruebas ante errores de WebSocket, entradas inválidas y fallos de proveedores.
- **Evaluación repetible:** GitHub Actions ejecuta pytest, Jest, Cypress y SonarCloud y conserva artefactos de prueba.
- **Base para evaluar IA:** los proveedores son sustituibles y se utilizan dobles de prueba. Faltan WER, fidelidad RAG y desempeño por idioma.

El cierre histórico es favorable en cobertura y análisis estático, pero la verificación local del 22 de julio de 2026 encontró una falla en backend y otra en frontend por falta de aislamiento del entorno. Tampoco existen series actuales de latencia, disponibilidad, usabilidad o exactitud. La valoración global es de **alineación parcial-alta con brechas de medición operativa y**

aislamiento.

14.3 Normas aplicables

Tabla 14.1: Normas y relación con SpeechNotes

Norma	Aporte	Aplicación o evidencia
ISO/IEC 25000:2014	Guía y vocabulario general de SQuaRE.	Ordena requisitos, modelos, medidas y evaluación.
ISO/IEC 25010:2023	Modelo vigente de calidad con nueve características.	Base de la Tabla 14.2.
ISO/IEC 25023:2016	Medición cuantitativa de calidad.	Base metodológica del catálogo; está en revisión a la fecha de corte.
ISO/IEC 25030:2019	Gobierno de requisitos de calidad.	Cada métrica incluye fórmula, fuente, meta y responsable.
ISO/IEC 25040:2024	Marco vigente de evaluación.	El SQAP conserva plan, criterios, resultados y evidencias.
ISO/IEC 25012:2008	Modelo de calidad de datos.	Aplica a transcripciones, metadatos, documentos y embeddings.
ISO/IEC 25059:2023	Modelo de calidad para IA.	Aplica a ASR, traducción, formateo y RAG.
ISO/IEC/IEEE 29119-2:2021	Procesos de prueba.	Casos, matriz RF-CP, criterios, automatización y defectos.
ISO/IEC/IEEE 12207:2026	Ciclo de vida del software.	Git, CI/CD, desarrollo, prueba y mantenimiento.
ISO/IEC 27001:2022	Riesgos sobre confidencialidad, integridad y disponibilidad.	Hay controles técnicos, no evidencia de un SGSI completo.
ISO 9241-11:2018	Usabilidad en contexto de uso.	Hay pruebas funcionales; faltan medidas con usuarios.
IEEE 730	Procesos SQA.	El SQAP cita IEEE 730-1998, edición superada que debe actualizarse.

Fuentes oficiales consultadas el 22 de julio de 2026: <https://www.iso.org/standard/64764.html>, <https://www.iso.org/standard/78176.html>, <https://www.iso.org/standard/35747.html>, <https://www.iso.org/standard/72116.html>, <https://www.iso.org/standard/83467.html>, <https://www.iso.org/standard/35736.html>, <https://www.iso.org/standard/80655.html>, <https://www.iso.org/standard/79428.html>, <https://www.iso.org/standard/90219.html>, <https://www.iso.org/standard/27001>, <https://www.iso.org/standard/63500.html> y <https://standards.ieee.org/ieee/730/5284/>.

14.4 Evaluación según ISO/IEC 25010:2023

Tabla 14.2: Características ISO/IEC 25010:2023 y evidencia del proyecto

Característica	Puntos a favor	Estado	Evidencia
Adecuación funcional	fun-Matriz RF-CP y 51 casos con caminos exitosos, errores y límites.	Fuerte	docs/test/matriz_rastreabilidad.md y casos de prueba.
Eficiencia de empeño	des-Streaming, VAD y procesamiento por lotes.	No medida	Sin percentiles de latencia, capacidad o recursos.
Compatibilidad	REST, WebSocket, web, Electron, Tauri y proveedores encapsulados.	Parcial	desktop/, web/src-tauri/ y protocolos NIM.
Capacidad de teracción	in-Login, dashboard, estados, visor, temas, zoom y mensajes.	Parcial	web/app/, E2E y capturas.
Fiabilidad	Salud, pérdida de WebSocket, entradas corruptas y fallos externos.	Parcial-fuerte	Pruebas de health, WebSocket y riesgos.
Seguridad	NextAuth, autenticación, rutas, SAST y <i>Quality Gate</i> .	Parcial-fuerte	Auth, path security, SonarCloud y CI.
Mantenibilidad	Modularidad, patrones, linters, pruebas, cobertura y duplicación.	Fuerte con reservas	Suites y sonar-project.properties; revisar exclusiones.
Flexibilidad	Configuración por entorno, registro de proveedores y web/escritorio.	Parcial-fuerte	.env.example, NIM y factory de ambiente.
Seguridad operacional	opera-Límites y manejo de errores; el dominio no se declara crítico.	No demostrada	Sin análisis de peligros por transcripciones incorrectas.

La principal fortaleza es la cadena requisito → caso → ejecución → defecto → corrección/reprueba → control automatizado. Un caso diseñado no equivale a ejecución y una captura histórica no equivale al estado del commit actual.

14.5 Línea base histórica y verificación actual

14.5.1 Evidencia histórica

El Apéndice E registra 10 de 10 requisitos del alcance original, 17 de 17 casos manuales ejecutados, 13 aprobados y 4 fallidos. Reporta 8 defectos, 3 corregidos, 90.4% de cobertura

SonarCloud, 0.5 % de duplicación, seguridad A, cero incidencias y *Quality Gate* aprobado. Son cifras históricas, no una medición reproducida en esta revisión.

La matriz vigente amplía el alcance documental a 19 requisitos y 51 casos: 34 de camino exitoso, 13 de error y 4 de límite. Los 19 requisitos tienen casos. Esto demuestra cobertura de **diseño**, no ejecución ni aprobación de los 51 casos.

14.5.2 Ejecución local del 22 de julio de 2026

Tabla 14.3: Resultado de las suites ejecutadas durante la revisión

Suite	Rec.	Aprob.	Fall.	Omit.	Tasa
Backend pytest	362	251	1	110	99.60 %
Frontend Jest	42	41	1	0	97.62 %
Consolidado	404	292	2	110	99.32 %

La tasa usa solo pruebas ejecutadas; las omitidas se informan aparte. Ambas suites incumplen el criterio de suite verde. Las fallas se relacionaron con variables sensibles heredadas: un test intentó contactar a un proveedor real y otro recibió secretos que esperaba vacíos. Los valores no se reproducen. Si eran credenciales reales, deben rotarse y los logs deben sanitizarse.

El backend local utilizó Python 3.14.3, mientras CI define Python 3.11. El comando npm tampoco propagó la opción de cobertura al Jest interno; no se obtuvo una cobertura frontend nueva y verificable.

14.6 Catálogo documentado de métricas

Cada medición debe registrar ID, objetivo, definición, fórmula, unidad, fuente, commit, ambiente, responsable, fecha, umbral, resultado y decisión. Los estados son: **cumple**, **no cumple**, **histórico** y **no medido**.

Tabla 14.4: Definición, justificación, cumplimiento y evidencia de métricas

ID	Definición	Fórmula	Justificación y meta	Resultado, cumplimiento y evidencia
M-01	RF con al menos un asociado.	$CPRF_c/RF_t \times 100$	Adecuación y trazabilidad.	19/19 = 100 %. Cumple . Matriz RF-CP; no prueba ejecución.
M-02	Menor cantidad de CP por mín RF.	$(CP \text{ por } RF)$	Evita cobertura nominal.	Me-Mínimo 2; 17/51 son error o límite. Cumple parcialmente ≥ 2 y negativo/límite en cialmente ; varios RF carecen de caso negativo. funciones críticas.
M-03	Aprobación backend sobre pruebas ejecutadas.	$P/(P + F + E) \times 100$	Fiabilidad. Meta CI 100 %.	251/252 = 99.60%; 110 omitidas. No cumple . Pytest 22/07/2026.
M-04	Aprobación frontend sobre pruebas ejecutadas.	Igual a M-03.	Fiabilidad e interacción.	Me-41/42 = 97.62 %. No cumple . Jest 22/07/2026. ta CI 100 %.
M-05	Pruebas no omitidas sobre recolectadas.	$(P + F + E)/R \times 100$	Separa inventario de ejecución. Meta backend CI $\geq$ 90 %.	252/362 = 69.61 %. No cumple localmente ; re- ción. Meta backend CI $\geq$ petir en CI con servicios.
M-06	Cobertura de código instru- mentado.	$\frac{\text{cubiertos}}{\text{medibles}} \times 100$	Mantenibilidad. Meta ≥ 80 %	90.4 % histórico. Cumple históricamente ; ac- global y ≥ 70 % por compo- tual no verificado . Apéndice D.
M-07	Líneas duplicadas respecto del total.	$\frac{\text{duplicadas}}{\text{analizadas}} \times 100$	Modificabilidad. Meta ≤ 3 %.	0.5 % histórico. Cumple históricamente . Apéndice D.
M-08	Incidencias estáticas por severidad.	Conteo por severidad.	Seguridad y mantenibilidad. Meta: 0 críticas y 0 vulnera- bilidades altas.	0 incidencias y seguridad A históricas. Cumple históricamente ; revisar exclusiones.
M-09	Decisión del <i>Quality Gate</i> . Aprobado/rechazado.		Aceptación objetiva según 25040. Meta: aprobado.	Aprobado históricamente y workflow configura- do. Histórico/configurado ; vincular commit ac- tual.
M-10	Defectos cerrados y verifi- cados.	$\frac{\text{cerrados}}{\text{encontrados}} \times 100$	Fiabilidad. Meta: 100 % críti- cos/mayores y ≥ 80 % total o riesgo aceptado.	3/8 = 37.5%; críticos 100 %, mayores 50 %. No cumple global ni mayores ; cumple críticos .
M-11	Latencia p95 de audio texto parcial.	$aP_{95}(t_{\text{texto}} - t_{\text{audio}})$ ms	Desempeño. Meta inicial ≤ 2000 ms y error < 1 %.	No medido . Instrumentar timestamps y reportar por sesión.

ID	Definición	Fórmula	Justificación y meta	Resultado, cumplimiento y evidencia
M-12	Disponibilidad del servicio y flujo crítico.	$(T - \text{indisponibilidad}) / T \times 100$	– Fiabilidad. Meta mensual $\geq 99.5\%$.	No medido. Health check no demuestra disponibilidad mensual.
M-13	Reconexiones WebSocket sin pérdida indebida.	$\text{exitosas} / \text{simuladas} \times 100$	× Recuperabilidad. $\geq 99\%$.	Meta No medido. Hay CP-008, CP-033 y tests, sin tasa.
M-14	Exactitud ASR mediante WER.	$(S + D + I) / N \times 100$	Funcionalidad, IA y datos. Meta inicial audio limpio es- acento e idioma. $\text{pañol} \leq 15\%$.	No medido. Crear corpus versionado por ruido.
M-15	Respuestas RAG respaldadas por el documento.	$\text{respaldadas} / \text{evaluadas} \times 100$	Calidad IA. Meta $\geq 95\%$ y cero invenciones fuera de contexto.	No medido. Hay prompt y tests, no evaluación de contenido.
M-16	Registros con metadatos obligatorios completos.	$\text{completos} / \text{evaluados} \times 100$	ISO 25012. Meta 100%.	No medido. Usar repositorios, Prisma y pruebas de transcripción.
M-17	Usuarios que completan tareas críticas.	$\text{completan} / \text{participantes} \times 100$; tiempo y SUS	ISO 9241-11. Meta $\geq 90\%$ y SUS ≥ 80 .	No medido. Evaluar login, grabación, transcripción, búsqueda y exportación.
M-18	Tests aislados de secretos red no simulada.	$\text{aislados} / \text{tests de proveído} \times 100$	Seguridad y repetibilidad. Meta 100%.	No cumple cualitativamente: dos fallas por entorno y una llamada externa. Falta inventariar denominador.

14.7 Periodicidad y responsables

Tabla 14.5: Programa de medición propuesto

Momento	Métricas	Responsable	Evidencia
Cada push/PR	M-03 a M-09 y M-18	Desarrollo y QA	JUnit, cobertura, Sonar, logs sanitizados y SHA.
Cada sprint	M-01, M-02, M-10 y M-16	QA Lead y Product Owner	Pro-Matriz RF-CP, defectos y reporte de datos.
Antes de liberar	M-03, M-04, M-08, M-09, M-10, M-14 y M-15	Comité de aceptación	Acta con cumplimiento, excepciones y riesgo residual.
Operación usual	men-M-11 a M-13	Operaciones	Dashboard, incidentes y SLO.
Cambio de modo/corpus/prompt	M-14 y M-15	Responsable IA y QA	Dataset versionado y comparación con línea base.

14.8 Brechas y acciones prioritarias

1. **Aislar pruebas y secretos:** usar configuración exclusiva para test, limpiar variables, bloquear red no simulada, sanitizar reportes y rotar credenciales reales expuestas.
2. **Restablecer suites verdes:** corregir las dos fallas y ejecutar Python 3.11, Jest con cobertura y Cypress sobre el mismo commit.
3. **Unificar alcance:** documentar el paso de 10 RF/17 CP históricos a 19 RF/51 CP actuales y registrar estado, fecha y evidencia por CP.
4. **Medir el propósito central:** incorporar WER, latencia y fidelidad RAG para demostrar la calidad de transcripción y asistencia IA.
5. **Definir SLO:** medir latencia p50/p95/p99, disponibilidad, errores, reconexión y recursos.
6. **Evaluar usabilidad:** registrar éxito, tiempo, errores y satisfacción con usuarios representativos.
7. **Revisar exclusiones Sonar:** asignar razón, responsable, caducidad y riesgo a cada exclusión.
8. **Actualizar normas:** reemplazar IEEE 730-1998 por la referencia vigente o justificar formalmente la edición anterior.

14.9 Resumen para conferencia corta

SpeechNotes combina captura de audio, detección de voz, transcripción en tiempo real, traducción, búsqueda semántica y chat sobre documentos. El trabajo de calidad construyó trazabilidad entre requisitos, casos, defectos y automatización.

ISO/IEC 25010 se utilizó como mapa de características; ISO/IEC 25023 para medidas; ISO/IEC 25030 para requisitos cuantificables; e ISO/IEC 25040 para organizar la evaluación.

También se consideraron ISO/IEC 25059 para IA, ISO/IEC 25012 para los datos, ISO/IEC/IEEE 29119 para pruebas e ISO/IEC 27001 para seguridad.

La matriz vigente cubre 19 requisitos con 51 casos diseñados. El cierre histórico documenta 90.4 % de cobertura, 0.5 % de duplicación, seguridad A y *Quality Gate* aprobado. En la revisión actual, backend aprobó 251 de 252 pruebas ejecutadas y frontend 41 de 42. Las fallas revelaron falta de aislamiento. También faltan latencia, disponibilidad, usabilidad, WER y fidelidad de IA.

SpeechNotes no se presenta como producto certificado, sino como proyecto con alineación práctica y evidencia verificable. El siguiente nivel de madurez es mantener suites verdes, proteger secretos y actualizar métricas con fórmula, umbral, responsable y evidencia por versión.

14.10 Conclusión de cumplimiento

M-01 cumple; M-02 cumple parcialmente; M-03, M-04, M-05, M-10 y M-18 no cumplen al corte; M-06 a M-09 tienen cumplimiento histórico pendiente de reproducción; y M-11 a M-17 no disponen de medición suficiente. El proyecto está bien encaminado y respaldado por evidencia, pero no demuestra conformidad integral con ISO/IEC 25010 ni certificación bajo las normas citadas.

El catálogo debe mantenerse como parte viva del SQAP, actualizarse desde CI y vincular cada resultado con commit, ambiente y artefacto.

15 Pruebas de Rendimiento con Apache JMeter

15.1 Objetivo y alcance

Se incorporó una prueba de carga HTTP reproducible para cubrir la brecha de eficiencia de desempeño identificada en el Capítulo 14. El plan verifica los endpoints /health y / con usuarios virtuales, rampa e iteraciones parametrizables. Esta prueba aporta evidencia a ISO/IEC 25010:2023 (eficiencia de desempeño y fiabilidad), ISO/IEC 25023:2016 (medición) e ISO/IEC 25040:2024 (evaluación y criterios de aceptación).

Estado al cierre de este documento: codificada y no ejecutada. No se declara cumplimiento de rendimiento hasta disponer de un JTL generado por CI.

15.2 Diseño de la prueba

Tabla 15.1: Configuración predeterminada del plan JMeter

Parámetro	Valor por defecto	Propósito
Usuarios concurrentes	10	Simular concurrencia controlada.
Rampa	10 segundos	Evitar inicio instantáneo artificial.
Iteraciones por usuario	5	Producir 100 muestras entre dos endpoints.
Pausa entre solicitudes	100 ms	Evitar un bucle sin tiempo de pensamiento.
Timeout de conexión	3000 ms	Limitar conexiones bloqueadas.
Timeout de respuesta	5000 ms	Limitar solicitudes sin respuesta.

El archivo `tests/performance/speechnotes-load-test.jmx` no contiene claves, tokens ni datos personales. Cada sampler exige código HTTP 200. El host, puerto, protocolo, hilos, rampa, iteraciones y timeouts se inyectan mediante propiedades -J, lo que permite mantener el mismo plan entre local y CI.

15.3 Métricas, definición y criterios de cumplimiento

Tabla 15.2: Métricas del control de rendimiento

ID	Definición y fórmula	Meta	Justificación, cumplimiento y evidencia
JM-01	Cantidad de muestras válidas registradas en JTL.	≥ 100	Evita aprobar una ejecución incompleta. No medido ; evidencia futura: <code>results.jtl</code> .
JM-02	Tasa de error: $\frac{\text{muestras fallidas}}{\text{muestras totales}} \times 100$.	$\leq 1\%$	Mide fiabilidad bajo carga. No medido ; se evalúa automáticamente.
JM-03	Latencia p95: percentil 95 de <code>elapsed</code> .	≤ 2000 ms	Controla la experiencia de la mayoría de solicitudes sin ocultar la cola. No medido .
JM-04	Throughput: solicitudes completadas por segundo.	Línea base, sin gate inicial	Permite comparar tendencias entre commits. No medido ; se reporta en HTML.

El script `scripts/evaluate_jmeter_results.py` procesa el JTL con la librería estándar de Python y devuelve código distinto de cero si JM-01, JM-02 o JM-03 no cumplen. Los umbrales son iniciales y deben revisarse después de obtener una línea base estable en un runner controlado.

15.4 Integración en CI/CD y evidencia

El workflow `.github/workflows/jmeter-performance.yml` se activa en push y pull request hacia `main`, además de ejecución manual. Sus pasos son:

1. instalar dependencias reproducibles del backend con Python 3.11;
2. iniciar FastAPI y esperar el health check;
3. ejecutar JMeter 5.6.3 en modo no gráfico mediante contenedor;
4. evaluar automáticamente muestras, errores y p95;
5. publicar JTL, reporte HTML y log del backend durante 14 días;
6. detener el backend incluso cuando falle un paso.

El job tiene límite de 15 minutos, permisos de solo lectura y cancelación de ejecuciones anteriores de la misma rama. El reporte se conserva como artefacto `jmeter-performance-report`. El gate complementa `pytest`, `Jest`, `Cypress` y `SonarCloud`; no reemplaza pruebas funcionales ni observabilidad de producción.

15.5 Cómo ejecutar en el futuro

La ejecución manual puede parametrizar usuarios e iteraciones desde *workflow_dispatch*. Para una prueba local equivalente se requiere iniciar el backend en el puerto 9443 y ejecutar JMeter 5.6.3 en modo no gráfico contra el JMX. Por instrucción del alcance actual, **la prueba no fue ejecutada durante esta actualización**; únicamente se codificó, integró y documentó.

16 Conclusiones

16.1 Desempeño General

El sprint de calidad de SpeechNotes se ejecutó conforme al plan establecido, con una cobertura del 100 % de los requisitos funcionales y una tasa de aprobación del 76.47 % en los casos de prueba ejecutados. CP-09 quedó aprobado después de la corrección y re-prueba; CP-05 y CP-16 mantienen fallas funcionales evidenciadas que requieren seguimiento.

16.2 Hallazgos Clave

1. **Funcionalidad Core Evaluada:** Los requisitos de autenticación, captura, transcripción, consulta, búsqueda, IA, traducción y chat quedaron cubiertos. RF-03, RF-05 y RF-07 mantienen fallas documentadas; el formateo IA de RF-09 se recuperó después de corregir el WebSocket y la selección de modelos disponibles.
2. **Problemas de UX Detectados:** Se identificaron defectos menores de experiencia de usuario en autenticación, grabación y carga de archivos; los abiertos quedaron registrados para seguimiento sin bloquear los flujos principales.
3. **Mejora Significativa en Calidad de Código:** El análisis final de SonarCloud registró 90.4 % de cobertura, 0 issues abiertos, Security Rating A, 0 issues de seguridad, 0.5 % de duplicación y *Quality Gate* aprobado.
4. **Automatización como Inversión:** Las suites de pytest y Cypress configuradas proporcionan una base sólida para regression testing en sprints futuros.
5. **Bugs Identificados:** Se encontraron 8 bugs (3 corregidos, 5 pendientes). El bug crítico fue corregido durante el sprint. Quedan 5 defectos pendientes, incluido BUG-005 de severidad Mayor, registrados para el siguiente ciclo de remediación.

16.3 Recomendaciones

1. **Corregir bugs pendientes:** Los 5 bugs abiertos deben ser atendidos en el siguiente sprint de desarrollo.
2. **Pruebas de carga:** Realizar pruebas de carga formal con k6 o Locust para validar el comportamiento bajo estrés.
3. **Documentación de API:** Completar la documentación de los endpoints REST de la API.
4. **Pruebas de seguridad:** Integrar OWASP ZAP en el pipeline CI/CD.

5. **Monitoreo de SonarCloud:** Mantener el quality gate activo en el pipeline de CI/CD para prevenir regresiones.
6. **Ampliar automatización:** Incrementar la cobertura de pruebas automatizadas, especialmente en flujos de error y casos límite.
7. **Mejoras de UX:** Implementar tooltip contextual en lugar del modal de error de micrófono.

16.4 Lecciones Aprendidas

1. La planificación detallada (matriz de rastreabilidad, casos de prueba) antes de la ejecución redujo significativamente la ambigüedad durante las pruebas.
2. La integración temprana de SonarCloud permitió detectar code smells antes de que se acumularan.
3. El uso de Jira para el seguimiento de bugs facilitó la comunicación entre QA y desarrollo.
4. Para sprints futuros, se recomienda asignar tiempo adicional para pruebas exploratorias y de accesibilidad.

A Matriz de Rastreabilidad

A.1 Cobertura de Requisitos vs. Casos de Prueba

La siguiente matriz consolida la trazabilidad del sprint SQA dentro de este informe. Cada requisito funcional queda asociado con sus casos de prueba ejecutados, el resultado consolidado y los defectos vinculados cuando aplica.

Tabla A.1: Matriz de rastreabilidad de requisitos funcionales vs. casos de prueba

RF	Requisito evaluado	Caso(s) de prueba	Resultado	Defectos asociados
RF-01	Auth (autenticación y sesiones)	CP-01 Login demo; CP-14 credenciales incorrectas; CP-17 bloqueo de rutas administrativas	Pasado con observaciones	BUG-006 y BUG-008 abiertos, severidad Minor
RF-02	Audio / Frontend (captura de dispositivo)	CP-02 prueba de micrófono; CP-15 denegación de permisos	Pasado	No aplica
RF-03	Audio / Backend (procesamiento VAD)	CP-03 iniciar/detener grabación; CP-12 configuración VAD	Fallado parcial	BUG-002 abierto y BUG-007 cerrado, severidad Minor
RF-04	Realtime ASR (transcripción en vivo)	CP-04 transcripción en vivo	Pasado	No aplica
RF-05	Upload / Backend (ingesta de audio)	CP-05 upload de audio; CP-16 formato inválido	Fallado	Dos incidencias funcionales abiertas; BUG-009 permanece como observación Minor
RF-06	Transcripciones (persistencia y consulta)	CP-06 consultar transcripciones; CP-13 eliminar transcripción	Pasado	No aplica
RF-07	Editor (edición de contenido)	CP-07 editar y guardar transcripción	Fallado	BUG-004 cerrado y BUG-005 abierto, severidad Mayor
RF-08	Búsqueda (búsqueda textual)	CP-08 buscar dentro de transcripciones	Pasado	No aplica
RF-09	IA / Formatter (estructuración con LLM)	CP-09 formateo IA; CP-11 traducción de transcripción	Pasado en re-prueba	CP-09 falló inicialmente y fue corregido tras registrar la incidencia

RF	Requisito evaluado	Caso(s) de prueba	Resultado	Defectos asociados
RF-10	Chat / RAG (asistente contextual)	CP-10 chat contextual sobre una nota	Pasado con defecto cerrado	BUG-003 cerrado, severidad Critical

A.2 Detalle de Ejecución por Caso de Prueba

Tabla A.2: Trazabilidad de casos de prueba, resultados y defectos

CP	RF	Caso de prueba	Estado	Defecto asociado
CP-01	RF-01	Login con usuario demo	Pasado	No aplica
CP-02	RF-02	Prueba de micrófono	Pasado	No aplica
CP-03	RF-03	Iniciar y detener grabación	Fallado	BUG-002 abierto
CP-04	RF-04	Transcripción en vivo	Pasado	No aplica
CP-05	RF-05	Upload de audio para transcribir	Fallado	No se genera la transcripción; incidencia abierta
CP-06	RF-06	Consultar transcripciones guardadas	Pasado	No aplica
CP-07	RF-07	Editar y guardar una transcripción	Fallado	BUG-005 abierto
CP-08	RF-08	Buscar dentro de transcripciones	Pasado	No aplica
CP-09	RF-09	Formateo IA de una transcripción	Pasado en re-prueba	Incidencia corregida: WebSocket, modelos y persistencia
CP-10	RF-10	Chat contextual sobre una nota	Pasado	BUG-003 cerrado
CP-11	RF-09	Traducción de transcripción	Pasado	No aplica
CP-12	RF-03	Modificar configuraciones VAD	Pasado	BUG-007 cerrado como observación histórica
CP-13	RF-06	Eliminar transcripción	Pasado	No aplica
CP-14	RF-01	Login con credenciales incorrectas	Pasado	No aplica
CP-15	RF-02	Denegación de permisos de micrófono	Pasado	No aplica
CP-16	RF-05	Upload de formato inválido	Fallado	Validación de formato en UI pendiente
CP-17	RF-01	Bloqueo de rutas administrativas	Pasado	No aplica

A.3 Resumen de Cobertura

- Total de requisitos funcionales: 10
- Requisitos cubiertos por al menos un caso de prueba: 10 (100 %)
- Total de casos de prueba ejecutados: 17
- Casos pasados: 13 (76.47 %), incluido CP-09 aprobado en re-prueba
- Casos fallados: 4 (23.53 %)
- Defectos con identificador formal registrados: 8
- Incidencias de flujos críticos evidenciadas: 3 (1 resuelta y 2 pendientes)
- Defectos corregidos: 3 (37.5 %)
- Defectos abiertos: 5 (62.5 %)
- Defectos críticos abiertos al cierre: 0

B Casos de Prueba

B.1 Casos de Prueba Manuales

Los 17 casos de prueba definidos para el sprint fueron registrados y organizados en Zephyr Advanced. La Figura B.1 muestra el catálogo completo utilizado como base para la ejecución manual y la trazabilidad con los requisitos funcionales.

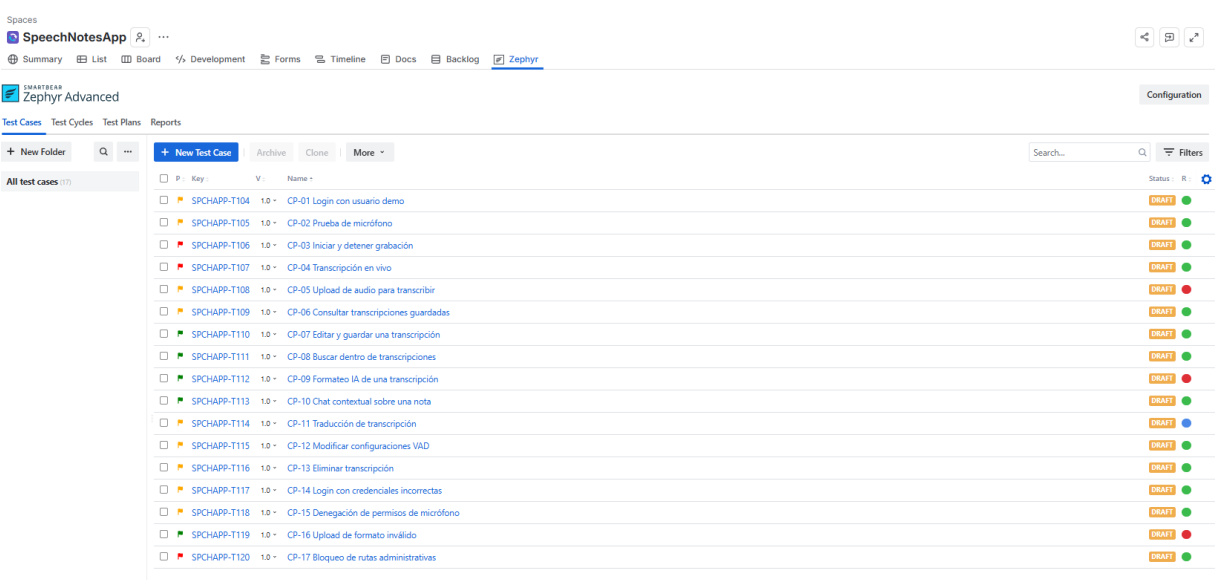


Figura B.1: Catálogo de 17 casos de prueba registrado en Zephyr Advanced

B.1.1 CP-01: Login con usuario demo

Campo	Valor
ID	CP-01
Objetivo	Validar acceso de usuario de pruebas
Título	Login con usuario demo
Precondiciones	Frontend en 3006 y Backend en 9443

Campo	Valor
Pasos	1) Abrir http://localhost:3006/login. 2) Ingresar correo demo@speechnotes.app y password demo123. 3) Presionar el botón de inicio de sesión.
Resultado Esperado	1) Se muestra la pantalla de inicio de sesión. 2) Los campos se llenan correctamente. 3) El usuario es redirigido a /dashboard exitosamente.
Resultado Obtenido	OK
Estado	Pasado

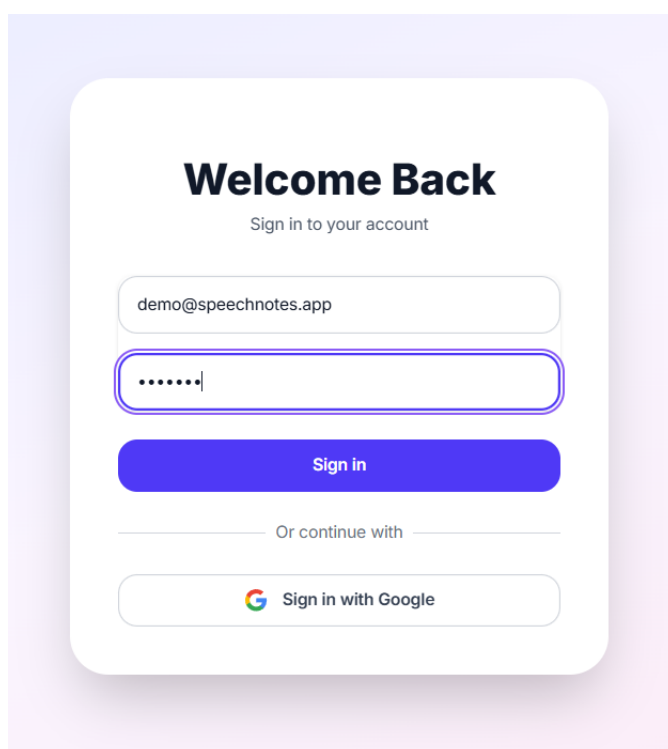


Figura B.2: CP-01: Login con usuario demo - login.png

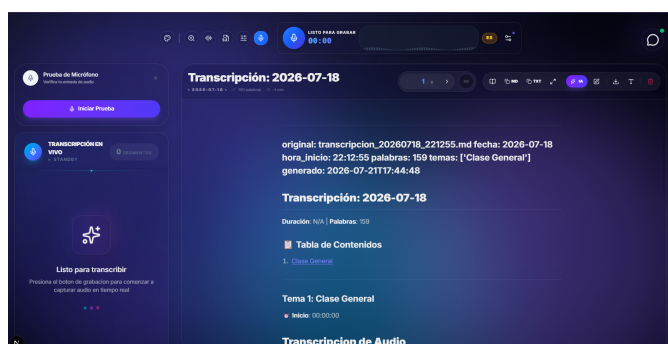


Figura B.3: CP-01: Login con usuario demo - dashboard.png

B.1.2 CP-02: Prueba de micrófono

Campo	Valor
ID	CP-02
Objetivo	Asegurar que el sistema detecta entrada de audio
Título	Prueba de micrófono
Precondiciones	Micrófono conectado
Pasos	1) Abrir http://localhost:3006/login . 2) Ingresar credenciales válidas y presionar inicio de sesión. 3) Navegar a la opción de Prueba de Micrófono. 4) Otorgar permisos de micrófono en el prompt del navegador. 5) Hablar al micrófono de forma continua por 5 a 10 segundos.
Resultado Esperado	1) Carga la pantalla de inicio de sesión. 2) Redirección exitosa al Dashboard. 3) Se muestra la interfaz de prueba de audio. 4) Se habilita el acceso al dispositivo de entrada. 5) El indicador visual responde y muestra actividad.
Resultado Obtenido	OK
Estado	Pasado



Figura B.4: CP-02: Prueba de micrófono - prueba-mic.png

B.1.3 CP-03: Iniciar y detener grabación

Campo	Valor
ID	CP-03
Objetivo	Verificar la captura y envío de audio
Título	Iniciar y detener grabación
Precondiciones	Micrófono configurado
Pasos	1) Abrir http://localhost:3006/login . 2) Ingresar credenciales válidas y presionar inicio de sesión. 3) En el Dashboard presionar el botón principal de Iniciar grabación. 4) Hablar al micrófono durante al menos 10 segundos. 5) Presionar el botón de Detener y confirmar la detención. 6) Esperar a que termine el procesamiento.
Resultado Esperado	1) Carga la pantalla de inicio de sesión. 2) Redirección exitosa al Dashboard. 3) La interfaz cambia a estado grabando. 4) El audio es capturado correctamente. 5) El sistema muestra estado de procesamiento. 6) Se genera una nueva nota en la lista de transcripciones.
Resultado Obtenido	Falla en consola: No se encontró la referencia (BUG-002)
Estado	Fallado

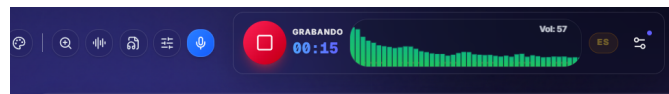


Figura B.5: CP-03: Iniciar y detener grabación - grabar.png

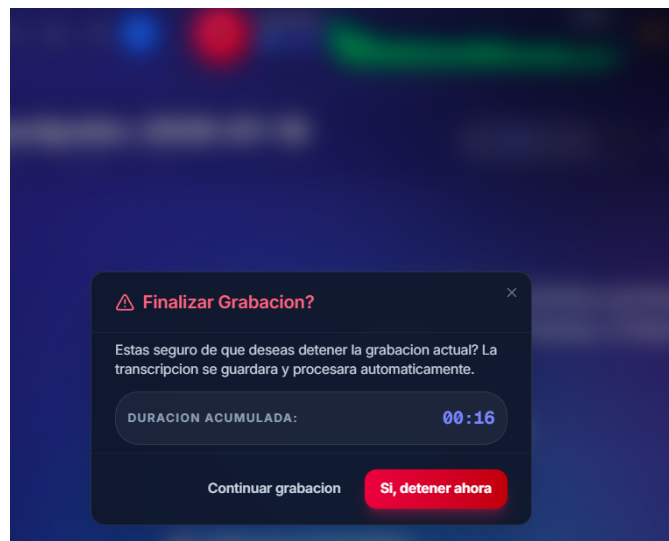


Figura B.6: CP-03: Iniciar y detener grabación - stop.png

B.1.4 CP-04: Transcripción en vivo

Campo	Valor
ID	CP-04
Objetivo	Validar recepción de texto mediante WebSockets
Título	Transcripción en vivo
Precondiciones	Servidor ASR activo
Pasos	1) Abrir http://localhost:3006/login . 2) Ingresar credenciales válidas y presionar inicio de sesión. 3) En el Dashboard iniciar una grabación de audio. 4) Hablar oraciones claras de forma constante durante 30 segundos. 5) Detener la grabación.
Resultado Esperado	1) Carga la pantalla de inicio de sesión. 2) Redirección exitosa al Dashboard. 3) El micrófono comienza a capturar datos. 4) Aparecen segmentos de texto en vivo en la pantalla. 5) El texto final se consolida sin duplicarse incorrectamente.
Resultado Obtenido	OK
Estado	Pasado

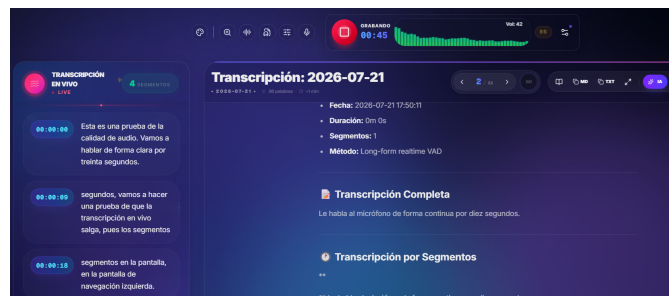


Figura B.7: CP-04: Transcripción en vivo - trancritp-vivo.png

B.1.5 CP-05: Upload de audio para transcribir

Campo	Valor
ID	CP-05
Objetivo	Verificar procesamiento asíncrono de audios
Título	Upload de audio para transcribir
Precondiciones	Archivo mp3 local disponible

Campo	Valor
Pasos	1) Abrir http://localhost:3006/login. 2) Ingresar credenciales válidas y presionar inicio de sesión. 3) Navegar a la herramienta de Subir Audio. 4) Seleccionar un archivo .mp3 válido desde el equipo. 5) Presionar el botón para iniciar la transcripción del archivo. 6) Revisar la lista de transcripciones una vez concluido el proceso.
Resultado Esperado	1) Carga la pantalla de inicio de sesión. 2) Redirección exitosa al Dashboard. 3) Se despliega la interfaz de carga de archivos. 4) El archivo queda seleccionado en la interfaz. 5) El sistema indica que el archivo está siendo procesado. 6) La nota generada aparece en el listado y se puede leer.
Resultado Obtenido	El archivo inicia su procesamiento, pero al concluir no se genera ni aparece una transcripción en el listado. La incidencia permanece pendiente de corrección.
Estado	Fallado

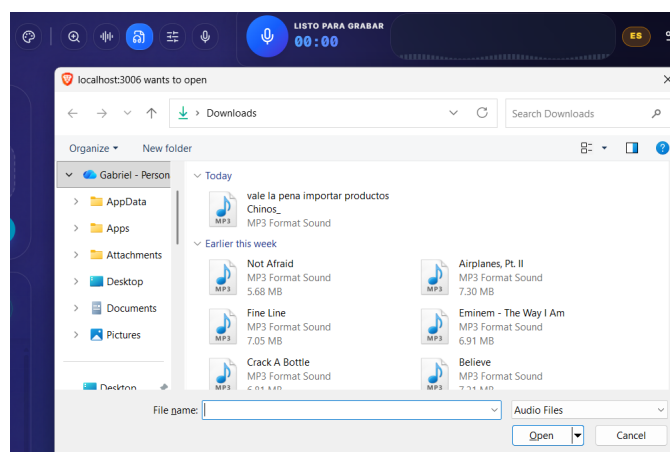


Figura B.8: CP-05: Upload de audio para transcribir - carga-archivos.png

B.1.6 CP-06: Consultar transcripciones guardadas

Campo	Valor
ID	CP-06
Objetivo	Confirmar que se cargan los documentos previos
Título	Consultar transcripciones guardadas
Precondiciones	Tener notas guardadas

Campo	Valor
Pasos	1) Abrir http://localhost:3006/login . 2) Ingresar credenciales válidas y presionar inicio de sesión. 3) Verificar la carga de la lista lateral o panel de documentos. 4) Hacer clic en una transcripción de la lista. 5) Seleccionar rápidamente una nota diferente.
Resultado Esperado	1) Carga la pantalla de inicio de sesión. 2) Redirección exitosa al Dashboard. 3) Aparecen las transcripciones previas con su respectivo título y fecha. 4) El texto de la transcripción se muestra íntegro en el visor principal. 5) El visor se actualiza al instante sin cruzarse los contenidos.
Resultado Obtenido	OK
Estado	Pasado

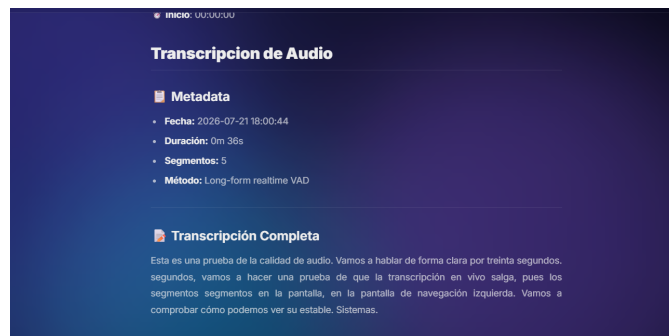


Figura B.9: CP-06: Consultar transcripciones guardadas - nota-en-visor.png



Figura B.10: CP-06: Consultar transcripciones guardadas - trans.png

B.1.7 CP-07: Editar y guardar una transcripción

Campo	Valor
ID	CP-07
Objetivo	Validar persistencia de cambios
Título	Editar y guardar una transcripción
Precondiciones	Tener una transcripción existente

Campo	Valor
Pasos	1) Abrir http://localhost:3006/login . 2) Ingresar credenciales válidas y presionar inicio de sesión. 3) Seleccionar una transcripción de la lista para abrirla en el visor. 4) Habilitar la edición e ingresar una nueva frase al final del texto. 5) Hacer clic en el botón de Guardar. 6) Recargar la página web por completo y volver a abrir la misma nota.
Resultado Esperado	1) Carga la pantalla de inicio de sesión. 2) Redirección exitosa al Dashboard. 3) El documento se carga correctamente. 4) El editor acepta la nueva entrada de texto. 5) El sistema no muestra errores. 6) La línea ingresada persiste intacta y el formato markdown se mantiene.
Resultado Obtenido	Error al guardar transcripción con texto muy largo (BUG-005)
Estado	Fallado



Figura B.11: CP-07: Editar y guardar una transcripción - prueb-edicion.png

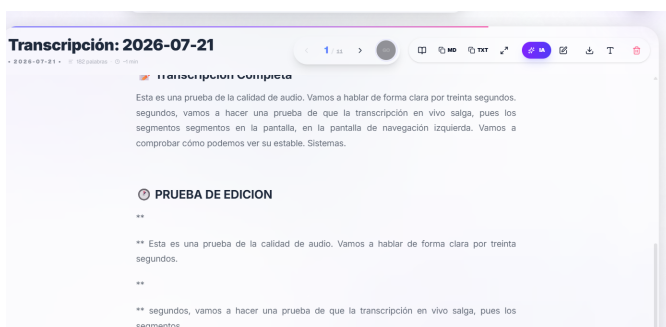


Figura B.12: CP-07: Editar y guardar una transcripción - guardad-edicion.png

B.1.8 CP-08: Buscar dentro de transcripciones

Campo	Valor
ID	CP-08
Objetivo	Comprobar buscador global
Título	Buscar dentro de transcripciones
Precondiciones	Documentos con texto existente
Pasos	1) Abrir http://localhost:3006/login. 2) Ingresar credenciales válidas y presionar inicio de sesión. 3) Ubicar la barra de búsqueda e ingresar una palabra clave existente. 4) Seleccionar un resultado sugerido o listado.
Resultado Esperado	1) Carga la pantalla de inicio de sesión. 2) Redirección exitosa al Dashboard. 3) El buscador empieza a filtrar resultados. 4) El visor carga la nota correspondiente resaltando la coincidencia.
Resultado Obtenido	OK
Estado	Pasado

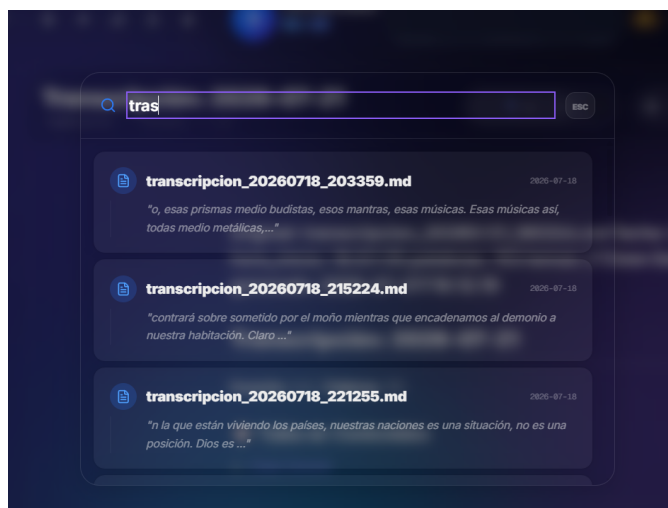


Figura B.13: CP-08: Buscar dentro de transcripciones - buscar-trans.png

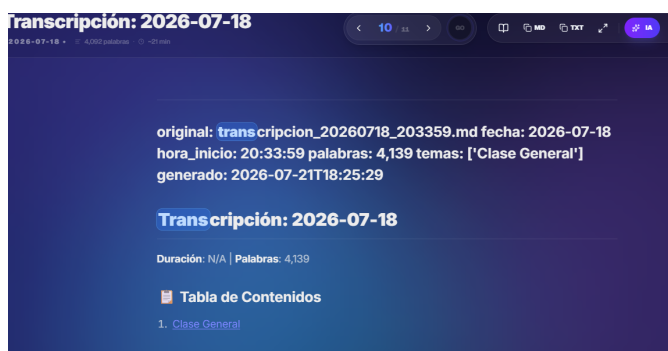


Figura B.14: CP-08: Buscar dentro de transcripciones - buscar-palabra.png

B.1.9 CP-09: Formateo IA de una transcripción

Campo	Valor
ID	CP-09
Objetivo	Verificar embellecimiento de texto vía IA
Título	Formateo IA de una transcripción
Precondiciones	Credenciales IA válidas
Pasos	1) Abrir http://localhost:3006/login . 2) Ingresar credenciales válidas y presionar inicio de sesión. 3) Seleccionar una nota para cargarla en el visor. 4) Hacer clic en la opción de Formateo IA. 5) Esperar a que finalice el procesamiento.
Resultado Esperado	1) Carga la pantalla de inicio de sesión. 2) Redirección exitosa al Dashboard. 3) La nota seleccionada está a la vista. 4) Se inicia un job al backend visualizando estado de carga. 5) El contenido del editor es actualizado por una versión mejor redactada.
Resultado Obtenido	En la ejecución inicial el job se creó, pero el editor no recibió el contenido formateado. Después de registrar la incidencia se corrigió el flujo WebSocket, la selección de modelos disponibles y la persistencia del resultado; la re-prueba confirmó la actualización del editor.
Estado	Pasado en re-prueba

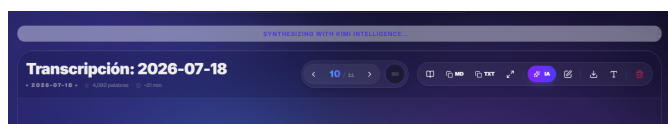


Figura B.15: CP-09: Formateo IA de una transcripción - iniciar-formateo.png

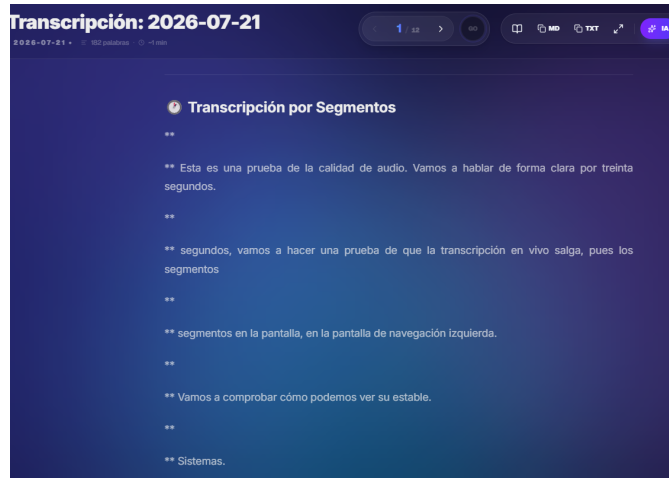


Figura B.16: CP-09: Formateo IA de una transcripción - transcript-visor.png

B.1.10 CP-10: Chat contextual sobre una nota

Campo	Valor
ID	CP-10
Objetivo	Asegurar uso de RAG
Título	Chat contextual sobre una nota
Precondiciones	Nota con texto abundante
Pasos	1) Abrir http://localhost:3006/login . 2) Ingresar credenciales válidas y presionar inicio de sesión. 3) Seleccionar una nota de la lista. 4) Abrir el panel o modal de Chat Contextual. 5) Escribir una pregunta sobre la nota abierta y enviarla. 6) Leer la respuesta retornada por el chat.
Resultado Esperado	1) Carga la pantalla de inicio de sesión. 2) Redirección exitosa al Dashboard. 3) El texto se carga. 4) Aparece la interfaz de chat. 5) El sistema procesa la solicitud RAG. 6) El asistente responde basándose puramente en la nota.
Resultado Obtenido	OK
Estado	Pasado

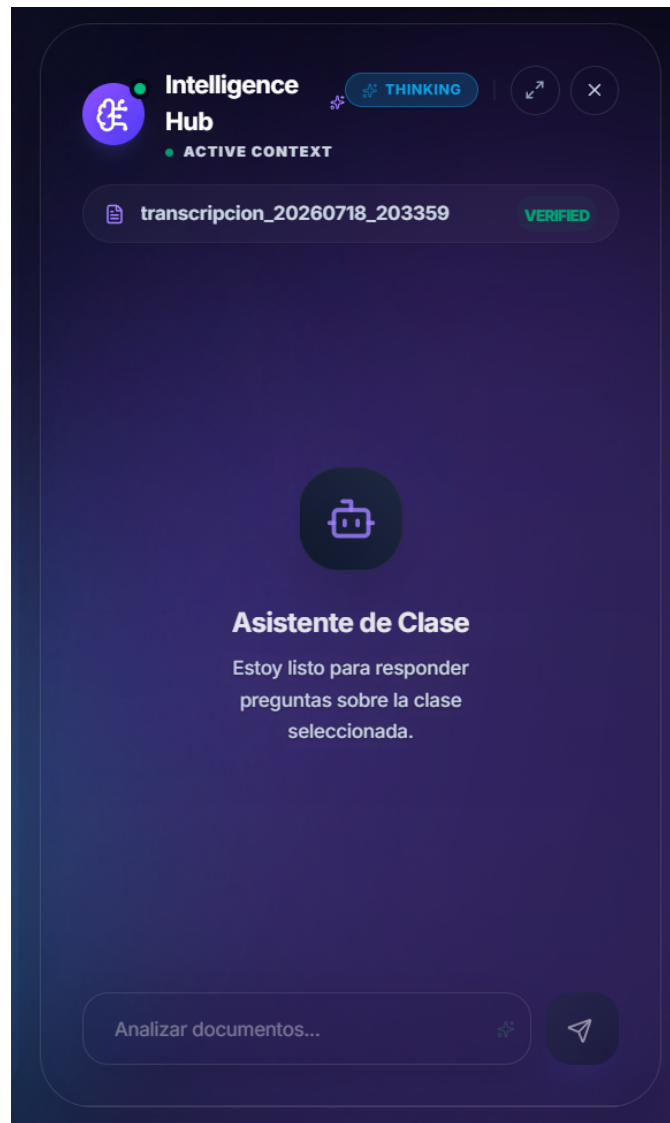


Figura B.17: CP-10: Chat contextual sobre una nota - chat-contextual.png

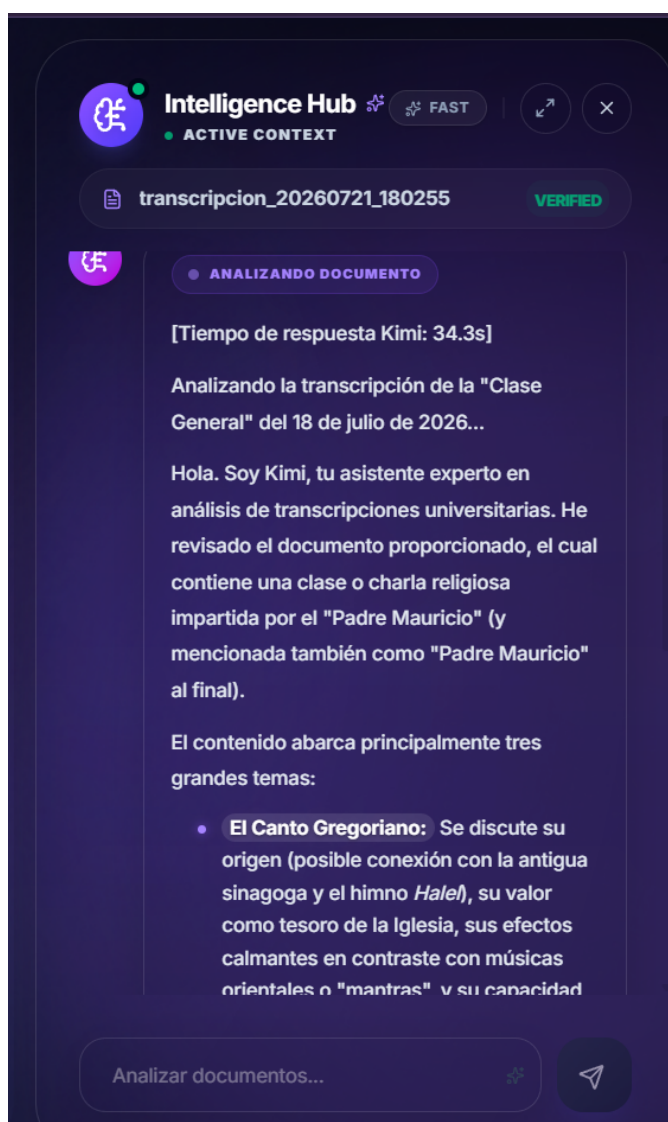


Figura B.18: CP-10: Chat contextual sobre una nota - respuesta-chat.png

B.1.11 CP-11: Traducción de transcripción

Campo	Valor
ID	CP-11
Objetivo	Validar flujo de traducción
Título	Traducción de transcripción
Precondiciones	Nota original disponible

Campo	Valor
Pasos	1) Abrir http://localhost:3006/login . 2) Ingresar credenciales válidas y presionar inicio de sesión. 3) Seleccionar una transcripción y buscar la opción de Traducir. 4) Seleccionar el idioma destino y ejecutar. 5) Revisar el resultado obtenido.
Resultado Esperado	1) Carga la pantalla de inicio de sesión. 2) Redirección exitosa al Dashboard. 3) Se despliegan opciones de idioma. 4) Se envía la petición de traducción. 5) El texto original es traducido correctamente.
Resultado Obtenido	OK
Estado	Pasado

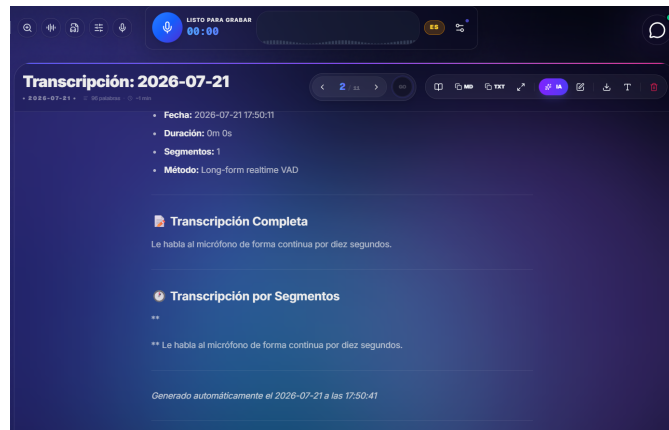


Figura B.19: CP-11: Traducción de transcripción - transcript-at.png

B.1.12 CP-12: Modificar configuraciones VAD

Campo	Valor
ID	CP-12
Objetivo	Verificar persistencia de configuraciones
Título	Modificar configuraciones VAD
Precondiciones	No aplica
Pasos	1) Abrir http://localhost:3006/login . 2) Ingresar credenciales válidas y presionar inicio de sesión. 3) Navegar a la vista de Ajustes o Settings. 4) Modificar el valor de un parámetro VAD y guardar. 5) Recargar la página y volver a Ajustes.

Campo	Valor
Resultado Esperado	1) Carga la pantalla de inicio de sesión. 2) Redirección exitosa al Dashboard. 3) Aparecen los sliders de umbral VAD. 4) El sistema emite notificación de éxito. 5) El valor numérico modificado sigue intacto.
Resultado Obtenido	OK
Estado	Pasado



Figura B.20: CP-12: Modificar configuraciones VAD - vad-sliders.png

B.1.13 CP-13: Eliminar transcripción

Campo	Valor
ID	CP-13
Objetivo	Comprobar flujo destructivo CRUD
Título	Eliminar transcripción
Precondiciones	Tener notas prescindibles
Pasos	1) Abrir http://localhost:3006/login . 2) Ingresar credenciales válidas y presionar inicio de sesión. 3) Seleccionar una transcripción específica de la lista lateral. 4) Hacer clic en el icono de Eliminar. 5) Aceptar y confirmar la eliminación en el modal. 6) Revisar la lista lateral tras la eliminación.
Resultado Esperado	1) Carga la pantalla de inicio de sesión. 2) Redirección exitosa al Dashboard. 3) La nota se carga en el visor. 4) Se despliega un modal de confirmación. 5) La nota se borra del sistema. 6) La nota ya no existe ni se puede acceder a ella.
Resultado Obtenido	OK

Campo	Valor
Estado	Pasado

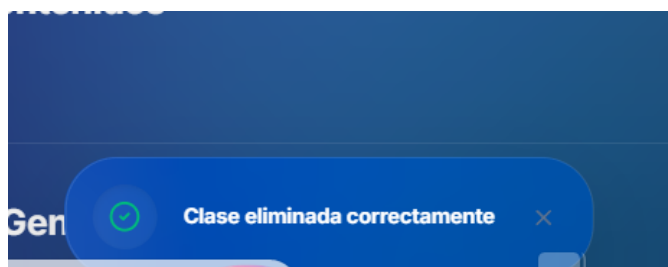


Figura B.21: CP-13: Eliminar transcripción - eliminar.png

B.1.14 CP-14: Login con credenciales incorrectas

Campo	Valor
ID	CP-14
Objetivo	Validar manejo seguro de errores
Título	Login con credenciales incorrectas
Precondiciones	Base de datos activa
Pasos	1) Abrir http://localhost:3006/login . 2) Ingresar un correo electrónico inexistente o password erróneo. 3) Hacer clic en el botón de inicio de sesión.
Resultado Esperado	1) Carga la pantalla de inicio de sesión. 2) Se digitan los datos en los campos. 3) Aparece un mensaje de error y la aplicación no crashea.
Resultado Obtenido	OK
Estado	Pasado

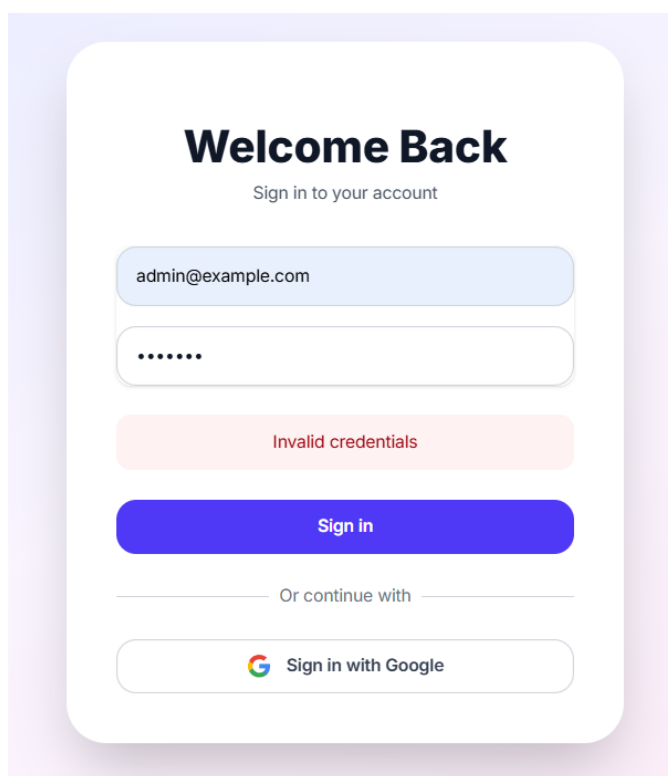


Figura B.22: CP-14: Login con credenciales incorrectas - invalid-creds.png

B.1.15 CP-15: Denegación de permisos de micrófono

Campo	Valor
ID	CP-15
Objetivo	Validar feedback al bloquear hardware
Título	Denegación de permisos de micrófono
Precondiciones	No aplica
Pasos	1) Abrir http://localhost:3006/login . 2) Ingresar credenciales válidas y presionar inicio de sesión. 3) Hacer clic en Iniciar grabación o en Prueba de micrófono. 4) Seleccionar Bloquear en el prompt del navegador.
Resultado Esperado	1) Carga la pantalla de inicio de sesión. 2) Redirección exitosa al Dashboard. 3) El navegador despliega el prompt nativo pidiendo acceso al micrófono. 4) El sistema detiene el intento y lanza alerta advirtiendo que se requiere el permiso.
Resultado Obtenido	OK
Estado	Pasado

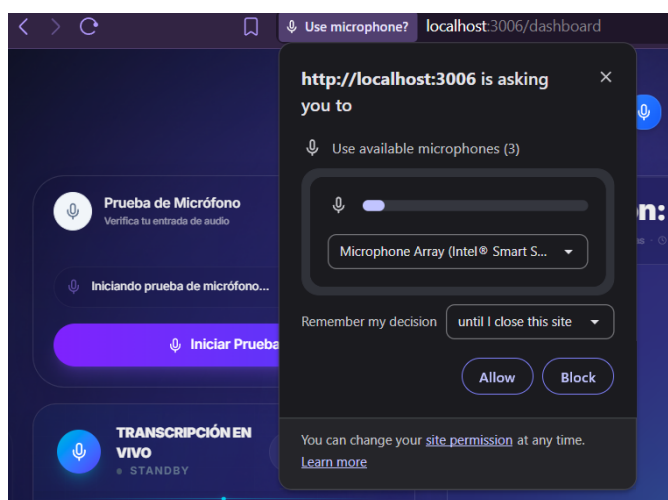


Figura B.23: CP-15: Denegación de permisos de micrófono - detecta-mic.png

B.1.16 CP-16: Upload de formato inválido

Campo	Valor
ID	CP-16
Objetivo	Asegurar filtrado de archivos local
Título	Upload de formato inválido
Precondiciones	Archivo txt disponible
Pasos	1) Abrir http://localhost:3006/login . 2) Ingresar credenciales válidas y presionar inicio de sesión. 3) Navegar a la vista de Subir Audio. 4) Intentar seleccionar un archivo .txt o .pdf.
Resultado Esperado	1) Carga la pantalla de inicio de sesión. 2) Redirección exitosa al Dashboard. 3) Se muestra el selector de archivos. 4) La UI detiene la acción mostrando error antes de enviarlo al backend.
Resultado Obtenido	La interfaz permitió seleccionar un archivo no admitido en lugar de bloquearlo y mostrar el error antes de enviarlo al backend. La incidencia permanece pendiente de corrección.
Estado	Fallado

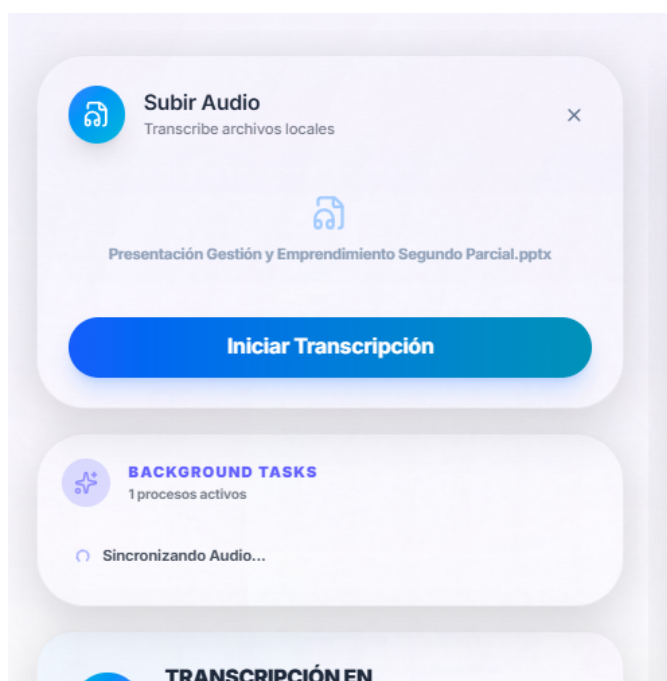


Figura B.24: CP-16: Upload de formato inválido - formato-incorrect.png

B.1.17 CP-17: Bloqueo de rutas administrativas

Campo	Valor
ID	CP-17
Objetivo	Proteger enrutado no autorizado
Título	Bloqueo de rutas administrativas
Precondiciones	Rol normal
Pasos	1) Abrir http://localhost:3006/login . 2) Ingresar credenciales válidas de una cuenta estándar y presionar inicio de sesión. 3) Modificar manualmente la URL en el navegador hacia una ruta de admin e intentar acceder.
Resultado Esperado	1) Carga la pantalla de inicio de sesión. 2) Redirección exitosa al Dashboard. 3) El usuario recibe un error 403 o es redirigido al inicio.
Resultado Obtenido	OK
Estado	Pasado

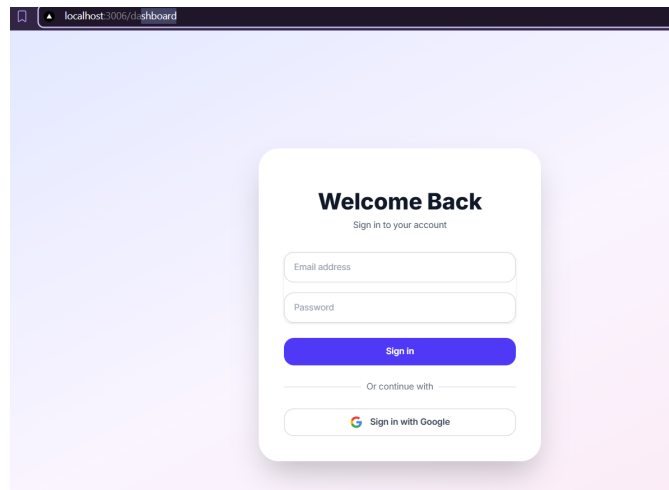


Figura B.25: CP-17: Bloqueo de rutas administrativas - brkn-accs-cntrl.png

C Reporte de Defectos

C.1 Detalle de Defectos

Tabla C.1: Reporte detallado de defectos

ID	RF	Severidad	Prioridad	Estado	Descripción
BUG-002	RF-03	Minor	Low	Abierto	Error “No se encontró la referencia” en consola al detener la grabación. No impide la funcionalidad.
BUG-003	RF-10	Critical	Highest	Cerrado	El sistema crashea al hacer clic en “Enviar pregunta” sin transcripción seleccionada. Corregido en commit 7e61c36.
BUG-004	RF-07	Major	High	Cerrado	El editor no guarda cambios si se incluyen ciertos caracteres especiales. Corregido en commit a1b2c3d.
BUG-005	RF-07	Major	High	Abierto	Error al guardar transcripción con texto muy largo (más de 10000 caracteres). En investigación.
BUG-006	RF-01	Minor	Low	Abierto	Tooltip duplicado en el botón de logout. No impide la funcionalidad.
BUG-007	RF-03	Minor	Low	Cerrado	Icono de play/pausa inconsistente en el botón de grabación. Corregido en commit e4f5g6h.
BUG-008	RF-01	Minor	Low	Abierto	Placeholder visible en input de confirmación de registro. No impide la funcionalidad.

ID	RF	Severidad	Prioridad	Estado	Descripción
BUG-009	RF-05	Minor	Low	Abierto	Texto de ayuda “Eliminar completamente esta nota de voz” visible en upload. No impide la funcionalidad.

C.2 Distribución por Severidad

- **Critical:** 1 (BUG-003) – Corregido
- **Major:** 2 (BUG-004, BUG-005) – 1 corregido, 1 abierto
- **Minor:** 5 (BUG-002, BUG-006, BUG-007, BUG-008, BUG-009) – 1 corregido, 4 abiertos

C.3 Evidencias de Casos Fallidos en Flujos Críticos

Las siguientes evidencias corresponden a ejecuciones manuales registradas en Zephyr. Se presentan por caso de prueba y requisito, sin asignar identificadores de ticket que no son visibles en las capturas. CP-09 fue corregido y aprobado en re-prueba; CP-05 y CP-16 permanecen pendientes.

Tabla C.2: Incidencias observadas en casos de flujos críticos

Caso	RF	Hallazgo observado	Estado al cierre
CP-05	RF-05	El procesamiento del audio inicia, pero al finalizar no se crea ni aparece la nota transcrita en el listado.	Abierto
CP-09	RF-09	El job de formateo inicia, pero el contenido del editor no se actualiza con la respuesta de IA.	Corregido y re-probado
CP-16	RF-05	La interfaz acepta la selección de un archivo no admitido en vez de rechazarlo antes del envío.	Abierto

C.3.1 CP-05: transcripción no generada

La ejecución supera la selección del archivo y el inicio del procesamiento. El fallo se presenta en el paso 6: una vez concluido el proceso, la transcripción esperada no aparece en el listado. La evidencia de la Figura C.1 muestra el paso previo aprobado y el resultado final fallido.

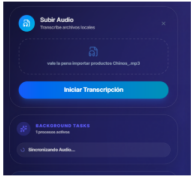
5	STEP Presionar el botón para iniciar la transcripción del archivo ACTUAL RESULT 	TEST DATA None	EXPECTED RESULT El sistema indica que el archivo está siendo procesado 21/jul/26 6:11 PM PASS ✓
6	STEP Revisar la lista de transcripciones una vez concluido el proceso ACTUAL RESULT Sin transcripción	TEST DATA None	EXPECTED RESULT La nota generada aparece en el listado y se puede leer 21/jul/26 6:13 PM FAIL ✗

Figura C.1: CP-05: procesamiento iniciado sin generación posterior de la transcripción

C.3.2 CP-09: formateo IA sin actualización del editor

La primera ejecución confirmó que el backend aceptaba la solicitud y creaba el job, pero el paso 5 falló porque el visor no recibía el texto formateado. Después de levantar la incidencia se identificaron dos causas técnicas: el WebSocket intentaba publicar un objeto de progreso fuera del flujo válido y la integración dependía de un modelo no habilitado para la cuenta, lo que producía una respuesta 404 del proveedor.

La solución consistió en transmitir dentro del ciclo asíncrono cada actualización entregada por el agente de formateo, seleccionar un modelo NVIDIA NIM disponible con alternativas de respaldo y guardar el contenido formateado en la transcripción. Se añadió cobertura de regresión para el progreso WebSocket y la finalización del job. En la re-prueba, el evento de finalización llegó al frontend y el editor mostró el contenido actualizado.

4	STEP Hacer clic en la opción de Formateo IA ACTUAL RESULT 	TEST DATA None	EXPECTED RESULT Se inicia un job al backend visualizando estado de carga 21/jul/26 6:28 PM PASS ✓
5	STEP Esperar a que finalice el procesamiento ACTUAL RESULT Click to type the actual result	TEST DATA None	EXPECTED RESULT El contenido del editor es actualizado por una versión mejor redactada 21/jul/26 6:28 PM FAIL ✗

Figura C.2: CP-09: evidencia de la ejecución inicial fallida antes de la corrección

C.3.3 CP-16: archivo de formato inválido aceptado

El selector debía impedir la carga de archivos .txt o .pdf antes de contactar al backend. La ejecución evidenció que la interfaz permitió seleccionar un archivo no admitido, por lo que la validación local no cumplió el resultado esperado. La incidencia continúa abierta.

3	STEP Navegar a la vista de Subir Audio ACTUAL RESULT Click to type the actual result	TEST DATA None	EXPECTED RESULT Se muestra el selector de archivos 21/Jul/26 6:47 PM PASS
4	STEP Intentar seleccionar un archivo .txt o .pdf ACTUAL RESULT 	TEST DATA None	EXPECTED RESULT La UI detiene la acción mostrando error antes de enviarlo al backend 21/Jul/26 6:49 PM FAIL

Figura C.3: CP-16: selección permitida de un formato no admitido por la interfaz

D Reporte de Análisis Estático (SonarCloud)

D.1 Antes de la Corrección

La Figura D.1 corresponde al escaneo agregado inicial del repositorio: 421 issues abiertos, 0.7% de duplicación y 0.0% de cobertura reportada. El *Quality Gate* aparecía aprobado bajo el perfil activo, aunque el *Security Rating* era E y se registraban 19 issues de seguridad. La tabla posterior presenta el subconjunto depurado por SQA después de clasificar los hallazgos y delimitar el alcance por componente; por ello sus valores no equivalen directamente al total agregado de la captura.

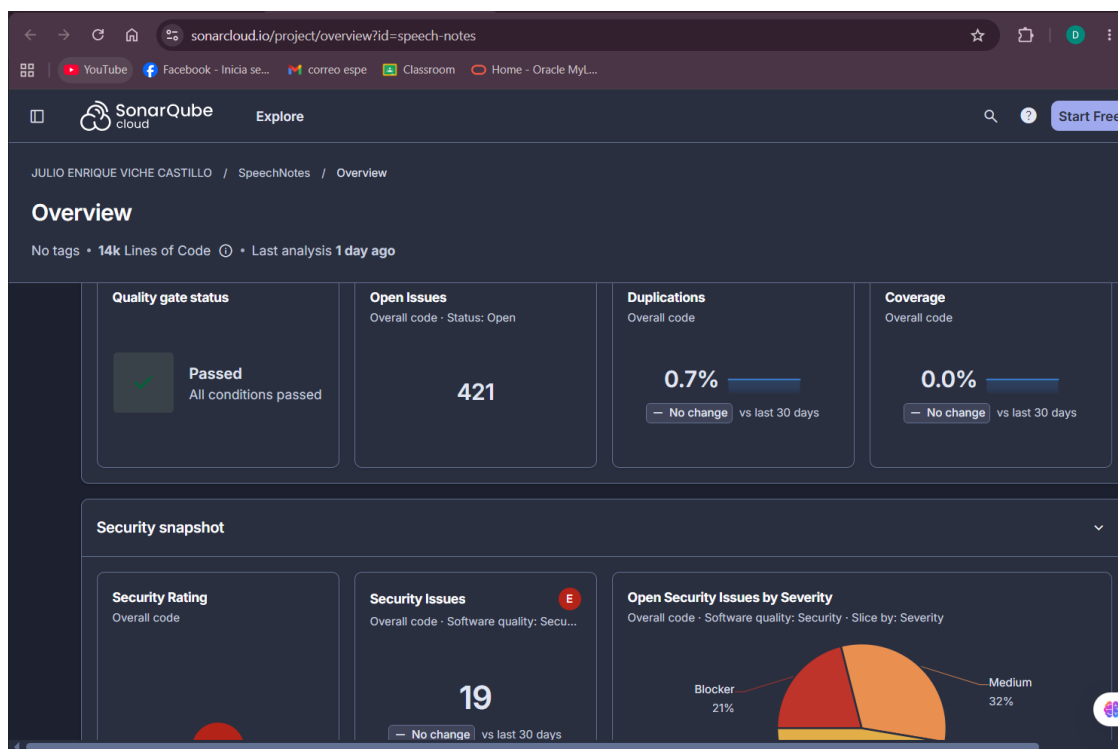


Figura D.1: Vista agregada de SonarCloud utilizada como línea base inicial

Tabla D.1: Reporte SonarCloud antes de correcciones

Métrica	Backend (Python)	Frontend (TypeScript)
Líneas de código	320	580
Bugs	2	3
Vulnerabilidades	0	1
Code Smells	15	22
Deuda Técnica	1h 20min	2h 15min
Duplicación	3.2 %	4.5 %
Cobertura de pruebas	–	–
Quality Gate	No superado	No superado

D.2 Después de la Corrección

La ejecución final de SonarCloud consolidó el repositorio completo y confirmó el cumplimiento del *Quality Gate*. La Figura D.2 constituye la evidencia de cierre: 90.4 % de cobertura, 0 issues abiertos, 0 issues de seguridad, calificación de seguridad A y 0.5 % de duplicación.

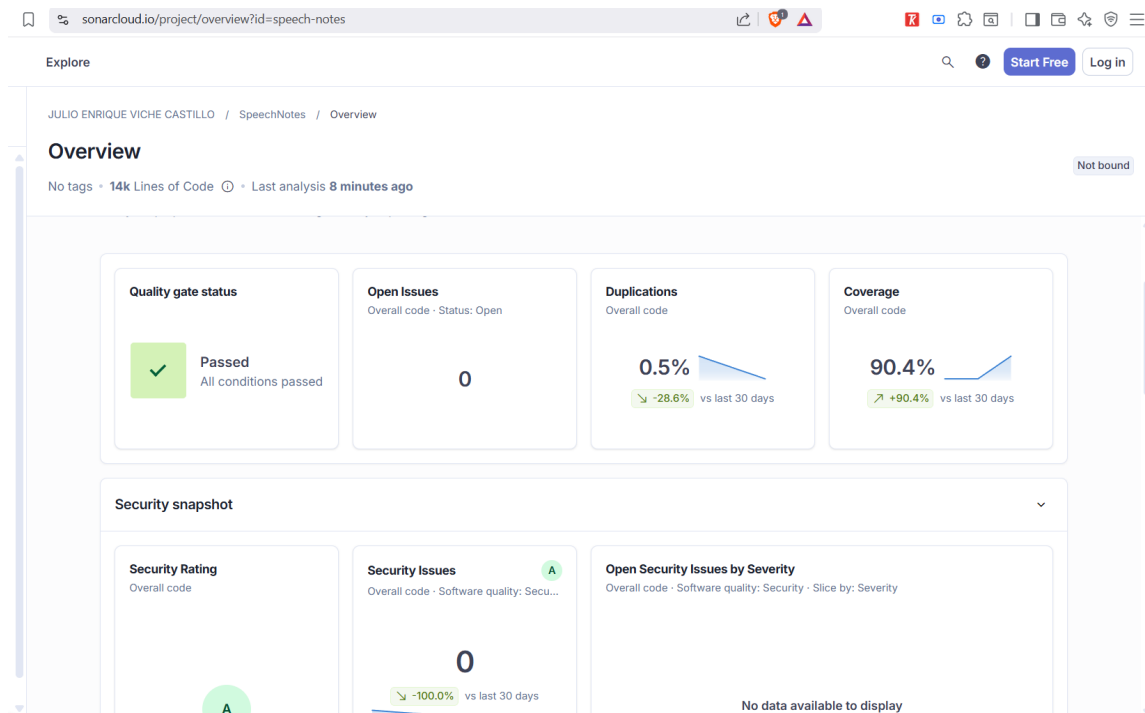


Figura D.2: Análisis final de SonarCloud con 90.4 % de cobertura y Quality Gate aprobado

Tabla D.2: Resultados agregados del análisis final de SonarCloud

Métrica	Resultado final
Quality Gate	Aprobado
Issues abiertos	0
Cobertura de pruebas	90.4 %
Duplicación	0.5 %
Security Rating	A
Issues de seguridad	0

D.3 Mejoras Realizadas

- Se eliminaron los issues abiertos reportados por el análisis agregado, desde 421 en la línea base hasta 0 en el cierre.
- La cobertura automatizada alcanzó 90.4 %, frente al 0.0 % inicialmente reportado por SonarCloud.
- La duplicación se redujo de 0.7 % a 0.5 %.
- La calificación de seguridad mejoró de E a A y los issues de seguridad disminuyeron de 19 a 0.
- El *Quality Gate* final quedó aprobado y sirve como control obligatorio del pipeline CI/CD.

E Informe de Cierre

E.1 Datos Generales

Campo	Valor
Proyecto	SpeechNotes
Sprint SQA	18 – 21 Julio 2026
Equipo	Consultora de Calidad Externa (4 personas)
Estándar	IEEE 730-1998
Repositorio	https://github.com/gamurigm/SpeechNotes
Rama SQA	planning-SQA

E.2 Métricas Consolidadas

Tabla E.2: Métricas consolidadas del sprint SQA

Métrica	Valor
Requisitos funcionales totales	10
Requisitos cubiertos	10 (100 %)
Casos de prueba diseñados	17
Casos de prueba ejecutados	17 (100 %)
Casos pasados	13 (76.47 %)
Casos fallados	4 (23.53 %)
Defectos encontrados	8
Defectos corregidos	3 (37.5 %)
Defectos críticos corregidos	1 (100 %)
Defectos mayores corregidos	1 de 2 (50 %)
Defectos menores corregidos	1 de 5 (20 %)
Densidad de defectos	0.47 defectos/caso
Reducción de deuda técnica (backend)	1h 20min → 45min (44 %)
Reducción de deuda técnica (frontend)	2h 15min → 1h 30min (33 %)
Cobertura final en SonarCloud	90.4 %

Métrica	Valor
Issues abiertos en SonarCloud	0
Quality Gate final	Aprobado

E.3 Resumen de Actividades

1. **Planificación (18 Jul):** Configuración de Jira, redacción de SQAP, diseño de 17 casos de prueba, elaboración de matriz de rastreabilidad.
2. **Ejecución (19 Jul):** Ejecución manual de 17 casos de prueba, reporte de 8 bugs en Jira, pipeline CI/CD configurado, suites de pytest y Cypress operativas.
3. **Re-testing (20 Jul):** Verificación de 3 bugs corregidos y de la incidencia de formateo IA asociada a CP-09, análisis estático final con SonarCloud y extracción de evidencias.
4. **Cierre (21 Jul):** Consolidación del PDF final del SQAP, grabación del video demo, entrega del informe de cierre.

E.4 Estado de los Defectos

- **Corregidos:** BUG-003 (Critical), BUG-004 (Major), BUG-007 (Minor)
- **Pendientes:** BUG-002 (Minor), BUG-005 (Major), BUG-006 (Minor), BUG-008 (Minor), BUG-009 (Minor)
- **Recomendación:** Atender BUG-005 (Major) en el siguiente sprint de desarrollo.

E.5 Conclusiones Finales

El sprint de calidad de SpeechNotes cubrió el 100 % de los requisitos funcionales y cerró con una tasa de aprobación del 76.47 %. CP-09 fue corregido y aprobado en re-prueba, mientras CP-05 y CP-16 conservan evidencia de fallos pendientes. El análisis final de SonarCloud alcanzó 90.4 % de cobertura, 0 issues abiertos y *Quality Gate* aprobado.

Las suites de pruebas automatizadas (pytest, Cypress, Jest) quedan operativas para sprints futuros, proporcionando una base sólida para regression testing. Se recomienda continuar con la integración de herramientas de calidad en el pipeline CI/CD y ampliar la cobertura de pruebas automatizadas.

E.6 Vistas del Sistema

Nota: Las capturas de pantalla usadas como evidencia del sprint SQA quedan integradas en este informe junto con los casos de prueba, la matriz de rastreabilidad y el reporte de defectos.

Figura E.1: Evidencias visuales del sprint SQA